

100% COMPLETE 202-SUBTOPIC EDITION

MySQL 8.0 & SQL Complete Master Textbook

A Full-Page A4 Reference Manual with Strict Multi-Line Indented Query Hierarchy, Hierarchical Index Table, Concepts, Executed Queries, Output Tables, and Pro Tips.



AUTHOR	SCOPE	LAYOUT	TOTAL BUDGET
TechBridge Engineering	202 Subtopics	Full-Height A4 Sheets	208 A4 Pages

Hierarchical Table of Contents

Organized by Main Topic Groups & Subtopic Commands

ID	SUBTOPIC NAME	PRACTICAL SYNTAX SUMMARY	A4 PAGE
■ 1. DATABASE FUNDAMENTALS & DDL			
#1	What is a Database?	-- DATABASE STORAGE ARCHITECTURE	Page 7
#2	What is RDBMS?	-- RELATIONAL INTEGRITY MODEL	Page 8
#3	MySQL Architecture	-- MYSQL CLIENT-SERVER PROTOCOL	Page 9
#4	CREATE DATABASE	CREATE DATABASE [IF NOT EXISTS] database_na...	Page 10
#5	DROP DATABASE	DROP DATABASE [IF EXISTS] database_name;	Page 11
#6	SHOW & USE DATABASES	SHOW DATABASES;	Page 12
#7	CREATE TABLE	CREATE TABLE table_name (	Page 13
#8	ALTER TABLE	ALTER TABLE table_name	Page 14
#9	DROP TABLE	DROP TABLE [IF EXISTS] table_name;	Page 15
#10	TRUNCATE TABLE	TRUNCATE TABLE table_name;	Page 16
#11	Constraints Overview	CREATE TABLE table_name (	Page 17
#12	NOT NULL Constraint	CREATE TABLE table_name (	Page 18
#13	UNIQUE Constraint	CREATE TABLE table_name (	Page 19
#14	PRIMARY KEY Constraint	CREATE TABLE table_name (	Page 20
#15	FOREIGN KEY Constraint	CREATE TABLE child_table (	Page 21
#16	CHECK Constraint	CREATE TABLE table_name (	Page 22
#17	DEFAULT Constraint	CREATE TABLE table_name (	Page 23
#18	AUTO_INCREMENT	CREATE TABLE table_name (	Page 24
#19	CREATE INDEX	CREATE INDEX index_name	Page 25
#20	DROP INDEX	DROP INDEX index_name	Page 26
#21	MySQL Views	CREATE VIEW view_name AS	Page 27
#22	SQL Injection Threats	-- UNSAFE CONCATENATED QUERY	Page 28
#23	Prepared Statements	PREPARE stmt FROM	Page 29
■ 2. MYSQL SQL & QUERIES			
#24	INSERT INTO	INSERT INTO table_name (	Page 30
#25	SELECT	SELECT	Page 31
#26	SELECT DISTINCT	SELECT DISTINCT	Page 32
#27	WHERE	SELECT	Page 33
#28	ORDER BY	SELECT	Page 34
#29	AND	SELECT	Page 35
#30	OR	SELECT	Page 36
#31	NOT	SELECT	Page 37
#32	NULL Values	SELECT	Page 38
#33	UPDATE	UPDATE table_name	Page 39
#34	DELETE	DELETE FROM table_name	Page 40
#35	LIMIT	SELECT	Page 41
#36	Aggregate Functions	SELECT	Page 42
#37	MIN()	SELECT	Page 43
#38	MAX()	SELECT	Page 44
#39	COUNT()	SELECT	Page 45
#40	SUM()	SELECT	Page 46

ID	SUBTOPIC NAME	PRACTICAL SYNTAX SUMMARY	A4 PAGE
■ 2. MYSQL SQL & QUERIES			
#41	AVG()	SELECT	Page 47
#42	LIKE	SELECT	Page 48
#43	Wildcards	SELECT	Page 49
#44	IN	SELECT	Page 50
#45	BETWEEN	SELECT	Page 51
#46	Aliases	SELECT	Page 52
#47	Joins	SELECT	Page 53
#48	INNER JOIN	SELECT	Page 54
#49	LEFT JOIN	SELECT	Page 55
#50	RIGHT JOIN	SELECT	Page 56
#51	CROSS JOIN	SELECT	Page 57
#52	Self Join	SELECT	Page 58
#53	UNION	SELECT col1 FROM table1	Page 59
#54	UNION ALL	SELECT col1 FROM table1	Page 60
#55	GROUP BY	SELECT	Page 61
#56	HAVING	SELECT	Page 62
#57	EXISTS	SELECT	Page 63
#58	ANY	SELECT	Page 64
#59	ALL	SELECT	Page 65
#60	INSERT SELECT	INSERT INTO archived_employees (	Page 66
#61	CASE	SELECT	Page 67
#62	Null Functions	SELECT	Page 68
#63	Stored Procedures	CREATE PROCEDURE GetDepartmentPayroll(	Page 69
#64	Comments & Operators	-- SINGLE LINE COMMENT	Page 70
■ 3. STRING FUNCTIONS			
#65	ASCII	-- FUNCTION SYNTAX	Page 71
#66	CHAR_LENGTH	-- FUNCTION SYNTAX	Page 72
#67	CHARACTER_LENGTH	-- FUNCTION SYNTAX	Page 73
#68	CONCAT	-- FUNCTION SYNTAX	Page 74
#69	CONCAT_WS	-- FUNCTION SYNTAX	Page 75
#70	FIELD	-- FUNCTION SYNTAX	Page 76
#71	FIND_IN_SET	-- FUNCTION SYNTAX	Page 77
#72	FORMAT	-- FUNCTION SYNTAX	Page 78
#73	INSERT	-- FUNCTION SYNTAX	Page 79
#74	INSTR	-- FUNCTION SYNTAX	Page 80
#75	LCASE	-- FUNCTION SYNTAX	Page 81
#76	LEFT	-- FUNCTION SYNTAX	Page 82
#77	LENGTH	-- FUNCTION SYNTAX	Page 83
#78	LOCATE	-- FUNCTION SYNTAX	Page 84
#79	LOWER	-- FUNCTION SYNTAX	Page 85
#80	LPAD	-- FUNCTION SYNTAX	Page 86

ID	SUBTOPIC NAME	PRACTICAL SYNTAX SUMMARY	A4 PAGE
■ 3. STRING FUNCTIONS			
#81	LTRIM	-- FUNCTION SYNTAX	Page 87
#82	MID	-- FUNCTION SYNTAX	Page 88
#83	POSITION	-- FUNCTION SYNTAX	Page 89
#84	REPEAT	-- FUNCTION SYNTAX	Page 90
#85	REPLACE	-- FUNCTION SYNTAX	Page 91
#86	REVERSE	-- FUNCTION SYNTAX	Page 92
#87	RIGHT	-- FUNCTION SYNTAX	Page 93
#88	RPAD	-- FUNCTION SYNTAX	Page 94
#89	RTRIM	-- FUNCTION SYNTAX	Page 95
#90	SPACE	-- FUNCTION SYNTAX	Page 96
#91	STRCMP	-- FUNCTION SYNTAX	Page 97
#92	SUBSTR	-- FUNCTION SYNTAX	Page 98
#93	SUBSTRING	-- FUNCTION SYNTAX	Page 99
#94	SUBSTRING_INDEX	-- FUNCTION SYNTAX	Page 100
#95	TRIM	-- FUNCTION SYNTAX	Page 101
#96	UCASE	-- FUNCTION SYNTAX	Page 102
#97	UPPER	-- FUNCTION SYNTAX	Page 103
■ 4. NUMERIC FUNCTIONS			
#98	ABS	-- FUNCTION SYNTAX	Page 104
#99	ACOS	-- FUNCTION SYNTAX	Page 105
#100	ASIN	-- FUNCTION SYNTAX	Page 106
#101	ATAN	-- FUNCTION SYNTAX	Page 107
#102	ATAN2	-- FUNCTION SYNTAX	Page 108
#103	AVG	-- FUNCTION SYNTAX	Page 109
#104	CEIL	-- FUNCTION SYNTAX	Page 110
#105	CEILING	-- FUNCTION SYNTAX	Page 111
#106	COS	-- FUNCTION SYNTAX	Page 112
#107	COT	-- FUNCTION SYNTAX	Page 113
#108	COUNT	-- FUNCTION SYNTAX	Page 114
#109	DEGREES	-- FUNCTION SYNTAX	Page 115
#110	DIV	-- FUNCTION SYNTAX	Page 116
#111	EXP	-- FUNCTION SYNTAX	Page 117
#112	FLOOR	-- FUNCTION SYNTAX	Page 118
#113	GREATEST	-- FUNCTION SYNTAX	Page 119
#114	LEAST	-- FUNCTION SYNTAX	Page 120
#115	LN	-- FUNCTION SYNTAX	Page 121
#116	LOG	-- FUNCTION SYNTAX	Page 122
#117	LOG10	-- FUNCTION SYNTAX	Page 123
#118	LOG2	-- FUNCTION SYNTAX	Page 124
#119	MAX	-- FUNCTION SYNTAX	Page 125
#120	MIN	-- FUNCTION SYNTAX	Page 126
#121	MOD	-- FUNCTION SYNTAX	Page 127
#122	PI	-- FUNCTION SYNTAX	Page 128
#123	POW	-- FUNCTION SYNTAX	Page 129
#124	POWER	-- FUNCTION SYNTAX	Page 130
#125	RADIANS	-- FUNCTION SYNTAX	Page 131
#126	RAND	-- FUNCTION SYNTAX	Page 132
#127	ROUND	-- FUNCTION SYNTAX	Page 133
#128	SIGN	-- FUNCTION SYNTAX	Page 134
#129	SIN	-- FUNCTION SYNTAX	Page 135
#130	SQRT	-- FUNCTION SYNTAX	Page 136
#131	SUM	-- FUNCTION SYNTAX	Page 137
#132	TAN	-- FUNCTION SYNTAX	Page 138
#133	TRUNCATE	-- FUNCTION SYNTAX	Page 139

ID	SUBTOPIC NAME	PRACTICAL SYNTAX SUMMARY	A4 PAGE
■ 5. DATE FUNCTIONS			
#134	ADDDATE	-- FUNCTION SYNTAX	Page 140
#135	ADDTIME	-- FUNCTION SYNTAX	Page 141
#136	CURDATE	-- FUNCTION SYNTAX	Page 142
#137	CURRENT_DATE	-- FUNCTION SYNTAX	Page 143
#138	CURRENT_TIME	-- FUNCTION SYNTAX	Page 144
#139	CURRENT_TIMESTAMP	-- FUNCTION SYNTAX	Page 145
#140	CURTIME	-- FUNCTION SYNTAX	Page 146
#141	DATE	-- FUNCTION SYNTAX	Page 147
#142	DATEDIFF	-- FUNCTION SYNTAX	Page 148
#143	DATE_ADD	-- FUNCTION SYNTAX	Page 149
#144	DATE_FORMAT	-- FUNCTION SYNTAX	Page 150
#145	DATE_SUB	-- FUNCTION SYNTAX	Page 151
#146	DAY	-- FUNCTION SYNTAX	Page 152
#147	DAYNAME	-- FUNCTION SYNTAX	Page 153
#148	DAYOFMONTH	-- FUNCTION SYNTAX	Page 154
#149	DAYOFWEEK	-- FUNCTION SYNTAX	Page 155
#150	DAYOFYEAR	-- FUNCTION SYNTAX	Page 156
#151	EXTRACT	-- FUNCTION SYNTAX	Page 157
#152	FROM_DAYS	-- FUNCTION SYNTAX	Page 158
#153	HOURL	-- FUNCTION SYNTAX	Page 159
#154	LAST_DAY	-- FUNCTION SYNTAX	Page 160
#155	LOCALTIME	-- FUNCTION SYNTAX	Page 161
#156	LOCALTIMESTAMP	-- FUNCTION SYNTAX	Page 162
#157	MAKEDATE	-- FUNCTION SYNTAX	Page 163
#158	MAKETIME	-- FUNCTION SYNTAX	Page 164
#159	MICROSECOND	-- FUNCTION SYNTAX	Page 165
#160	MINUTE	-- FUNCTION SYNTAX	Page 166
#161	MONTH	-- FUNCTION SYNTAX	Page 167
#162	MONTHNAME	-- FUNCTION SYNTAX	Page 168
#163	NOW	-- FUNCTION SYNTAX	Page 169
#164	PERIOD_ADD	-- FUNCTION SYNTAX	Page 170
#165	PERIOD_DIFF	-- FUNCTION SYNTAX	Page 171
#166	QUARTER	-- FUNCTION SYNTAX	Page 172
#167	SECOND	-- FUNCTION SYNTAX	Page 173
#168	SEC_TO_TIME	-- FUNCTION SYNTAX	Page 174
#169	STR_TO_DATE	-- FUNCTION SYNTAX	Page 175
#170	SUBDATE	-- FUNCTION SYNTAX	Page 176
#171	SUBTIME	-- FUNCTION SYNTAX	Page 177
#172	SYSDATE	-- FUNCTION SYNTAX	Page 178
#173	TIME	-- FUNCTION SYNTAX	Page 179
#174	TIME_FORMAT	-- FUNCTION SYNTAX	Page 180
#175	TIME_TO_SEC	-- FUNCTION SYNTAX	Page 181

ID	Subtopic Name	Practical Syntax Summary	A4 Page
■ 5. Date Functions			
#176	TIMEDIFF	-- FUNCTION SYNTAX	Page 182
#177	TIMESTAMP	-- FUNCTION SYNTAX	Page 183
#178	TO_DAYS	-- FUNCTION SYNTAX	Page 184
#179	WEEK	-- FUNCTION SYNTAX	Page 185
#180	WEEKDAY	-- FUNCTION SYNTAX	Page 186
#181	WEEKOFYEAR	-- FUNCTION SYNTAX	Page 187
#182	YEAR	-- FUNCTION SYNTAX	Page 188
#183	YEARWEEK	-- FUNCTION SYNTAX	Page 189
■ 6. Advanced Functions			
#184	BIN	-- FUNCTION SYNTAX	Page 190
#185	BINARY	-- FUNCTION SYNTAX	Page 191
#186	CASE	-- FUNCTION SYNTAX	Page 192
#187	CAST	-- FUNCTION SYNTAX	Page 193
#188	COALESCE	-- FUNCTION SYNTAX	Page 194
#189	CONNECTION_ID	-- FUNCTION SYNTAX	Page 195
#190	CONV	-- FUNCTION SYNTAX	Page 196
#191	CONVERT	-- FUNCTION SYNTAX	Page 197
#192	CURRENT_USER	-- FUNCTION SYNTAX	Page 198
#193	DATABASE	-- FUNCTION SYNTAX	Page 199
#194	IF	-- FUNCTION SYNTAX	Page 200
#195	IFNULL	-- FUNCTION SYNTAX	Page 201
#196	ISNULL	-- FUNCTION SYNTAX	Page 202
#197	LAST_INSERT_ID	-- FUNCTION SYNTAX	Page 203
#198	NULLIF	-- FUNCTION SYNTAX	Page 204
#199	SESSION_USER	-- FUNCTION SYNTAX	Page 205
#200	SYSTEM_USER	-- FUNCTION SYNTAX	Page 206
#201	USER	-- FUNCTION SYNTAX	Page 207
#202	VERSION	-- FUNCTION SYNTAX	Page 208

What is a Database?

 CONCEPT NOTES & DEFINITION

An organized collection of structured data stored electronically in a computer system for fast search, insertion, and manipulation.

 FUNCTION / COMMAND SYNTAX

```
-- DATABASE STORAGE ARCHITECTURE
FILES -> SCHEMAS -> TABLES -> ROWS & COLUMNS
```

 SAMPLE INPUT DATA TABLE

System_Type	Definition
Database	Persistent Structured Data Storage

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
'Database' AS System_Type,
'Persistent Structured Data Storage' AS Definition;
```

 EXECUTION OUTPUT RESULT TABLE

System_Type	Definition
Database	Persistent Structured Data Storage

 STEP-BY-STEP LOGIC

- 1. Identifies purpose of databases vs raw text files.
- 2. Demonstrates persistence and structured query interface.

 PRO TIP & PITFALL

Databases use indexing and transaction logs (ACID) to ensure data durability and high performance under concurrency.

What is RDBMS?

 CONCEPT NOTES & DEFINITION

Relational Database Management System - a database model based on tables (relations) with rows and columns, linked by key relationships.

 FUNCTION / COMMAND SYNTAX

```
-- RELATIONAL INTEGRITY MODEL
PRIMARY KEY (Parent) <---> FOREIGN KEY (Child)
```

 SAMPLE INPUT DATA TABLE

Architecture	Core_Feature
RDBMS	Tables linked by PK/FK constraints

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  'RDBMS' AS Architecture,
  'Tables linked by PK/FK constraints' AS Core_Feature;
```

 EXECUTION OUTPUT RESULT TABLE

Architecture	Core_Feature
RDBMS	Tables linked by PK/FK constraints

 STEP-BY-STEP LOGIC

1. Organizes data into tabular entities. 2. Uses primary and foreign keys to enforce relational integrity across tables.

 PRO TIP & PITFALL

Popular RDBMS solutions include MySQL, PostgreSQL, Oracle, and Microsoft SQL Server.

MySQL Architecture

★ CONCEPT NOTES & DEFINITION

An open-source relational database management system using a client-server model and pluggable storage engines like InnoDB.

⚙️ FUNCTION / COMMAND SYNTAX

```
-- MYSQL CLIENT-SERVER PROTOCOL
CLIENT (Port 3306) <---> SERVER ENGINE (InnoDB)
```

📊 SAMPLE INPUT DATA TABLE

MySQL_Version	Default_Engine
8.0.35-TechBridge	InnoDB

🖨️ EXECUTED PRACTICAL SQL QUERY

```
SELECT
  VERSION() AS MySQL_Version,
  @@storage_engine AS Default_Engine;
```

📄 EXECUTION OUTPUT RESULT TABLE

MySQL_Version	Default_Engine
8.0.35-TechBridge	InnoDB

💡 STEP-BY-STEP LOGIC

- 1. Queries server version and default storage engine.
- 2. Confirms InnoDB transactional support.

⚠️ PRO TIP & PITFALL

InnoDB is the default storage engine in MySQL 8.0, offering ACID compliance and foreign key constraints.

CREATE DATABASE

 CONCEPT NOTES & DEFINITION

Creates a new named database schema space on the MySQL server.

 FUNCTION / COMMAND SYNTAX

```
CREATE DATABASE [IF NOT EXISTS] database_name
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

 SAMPLE INPUT DATA TABLE

Status	Message
SUCCESS	Database techbridge_db created successfully.

 EXECUTED PRACTICAL SQL QUERY

```
CREATE DATABASE IF NOT EXISTS techbridge_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

 EXECUTION OUTPUT RESULT TABLE

Status	Message
SUCCESS	Database techbridge_db created successfully.

 STEP-BY-STEP LOGIC

- 1. Checks if techbridge_db exists.
- 2. Allocates new database schema directory on disk server.

 PRO TIP & PITFALL

Always use IF NOT EXISTS to prevent error crashes during automated deployment scripts.

DROP DATABASE

 CONCEPT NOTES & DEFINITION

Permanently deletes a database schema along with all its contained tables and records.

 FUNCTION / COMMAND SYNTAX

```
DROP DATABASE [IF EXISTS] database_name;
```

 SAMPLE INPUT DATA TABLE

Status	Message
SUCCESS	Database test_staging_db deleted.

 EXECUTED PRACTICAL SQL QUERY

```
DROP DATABASE IF EXISTS test_staging_db;
```

 EXECUTION OUTPUT RESULT TABLE

Status	Message
SUCCESS	Database test_staging_db deleted.

 STEP-BY-STEP LOGIC

- 1. Checks for test_staging_db existence.
- 2. Removes schema directory and all internal data.

 PRO TIP & PITFALL

WARNING: DROP DATABASE is irreversible!
Always take backups before executing in production environments.

SHOW & USE DATABASES

 CONCEPT NOTES & DEFINITION

Lists available database schemas and switches active database context for query execution.

 FUNCTION / COMMAND SYNTAX

```
SHOW DATABASES;  
USE database_name;
```

 SAMPLE INPUT DATA TABLE

Database_Name	Context_Status
techbridge_db	ACTIVE_CONTEXT

 EXECUTED PRACTICAL SQL QUERY

```
SHOW DATABASES;  
USE techbridge_db;
```

 EXECUTION OUTPUT RESULT TABLE

Database_Name	Context_Status
techbridge_db	ACTIVE_CONTEXT

 STEP-BY-STEP LOGIC

1. Scans server schemas. 2. Sets techbridge_db as the default schema context for session.

 PRO TIP & PITFALL

You must execute USE database_name before creating or querying tables in CLI tools.

CREATE TABLE

★ CONCEPT NOTES & DEFINITION

Defines a new table structure including column names, data types, and constraint definitions.

⚙️ FUNCTION / COMMAND SYNTAX

```
CREATE TABLE table_name (  
    col1 DATATYPE PRIMARY KEY,  
    col2 DATATYPE NOT NULL  
);
```

📊 SAMPLE INPUT DATA TABLE

Status	Table_State
SUCCESS	Table 'employees' defined with 4 columns.

📄 EXECUTED PRACTICAL SQL QUERY

```
CREATE TABLE employees (  
    emp_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_name VARCHAR(100) NOT NULL,  
    department VARCHAR(50) NOT NULL,  
    salary DECIMAL(10,2) NOT NULL  
);
```

📄 EXECUTION OUTPUT RESULT TABLE

Status	Table_State
SUCCESS	Table 'employees' defined with 4 columns.

💡 STEP-BY-STEP LOGIC

- 1. Allocates storage definition for table 'employees'.
- 2. Binds emp_id as integer auto-increment primary key.

⚠️ PRO TIP & PITFALL

Define explicit column sizes (e.g. VARCHAR(100)) to optimize storage allocation in InnoDB.

ALTER TABLE

 CONCEPT NOTES & DEFINITION

Modifies an existing table schema by adding, dropping, or modifying column definitions.

 FUNCTION / COMMAND SYNTAX

```
ALTER TABLE table_name
  ADD COLUMN new_col DATATYPE [CONSTRAINT];
```

 SAMPLE INPUT DATA TABLE

Status	Schema_Change
SUCCESS	Column 'email' added to 'employees'.

 EXECUTED PRACTICAL SQL QUERY

```
ALTER TABLE employees
  ADD COLUMN email VARCHAR(150) UNIQUE AFTER emp_name;
```

 EXECUTION OUTPUT RESULT TABLE

Status	Schema_Change
SUCCESS	Column 'email' added to 'employees'.

 STEP-BY-STEP LOGIC

- 1. Inspects existing table 'employees'.
- 2. Inserts 'email' column with UNIQUE constraint after emp_name.

 PRO TIP & PITFALL

Modifying columns on massive tables with millions of rows can lock tables; use online DDL tools for live DBs.

DROP TABLE

★ CONCEPT NOTES & DEFINITION

Completely erases a table structure and all stored records from the database.

⚙️ FUNCTION / COMMAND SYNTAX

```
DROP TABLE [IF EXISTS] table_name;
```

📊 SAMPLE INPUT DATA TABLE

Status	Message
SUCCESS	Table temp_payroll_draft dropped.

🖨️ EXECUTED PRACTICAL SQL QUERY

```
DROP TABLE IF EXISTS temp_payroll_draft;
```

📋 EXECUTION OUTPUT RESULT TABLE

Status	Message
SUCCESS	Table temp_payroll_draft dropped.

💡 STEP-BY-STEP LOGIC

- 1. Verifies table presence.
- 2. Erases data files and structural metadata.

⚠️ PRO TIP & PITFALL

Use DROP TABLE IF EXISTS to safely run cleanup scripts without error interrupts.

TRUNCATE TABLE

 CONCEPT NOTES & DEFINITION

Removes all rows from a table rapidly by dropping and re-creating the table structure.

 FUNCTION / COMMAND SYNTAX

```
TRUNCATE TABLE table_name;
```

 SAMPLE INPUT DATA TABLE

Status	Rows_Affected
SUCCESS	All rows truncated, AUTO_INCREMENT reset to 1.

 EXECUTED PRACTICAL SQL QUERY

```
TRUNCATE TABLE audit_logs;
```

 EXECUTION OUTPUT RESULT TABLE

Status	Rows_Affected
SUCCESS	All rows truncated, AUTO_INCREMENT reset to 1.

 STEP-BY-STEP LOGIC

1. Unlinks table data pages. 2. Resets auto-increment counter to initial value.

 PRO TIP & PITFALL

TRUNCATE is faster than DELETE because it does not scan individual rows or generate row-level undo logs.

Constraints Overview

 CONCEPT NOTES & DEFINITION

Enforces data domain integrity, uniqueness, and relational consistency rules across table columns.

 FUNCTION / COMMAND SYNTAX

```
CREATE TABLE table_name (  
    col1 DATATYPE PRIMARY KEY,  
    col2 DATATYPE NOT NULL UNIQUE  
);
```

 SAMPLE INPUT DATA TABLE

Status	Integrity_Level
ACTIVE	Primary Key & Check constraints enforced.

 EXECUTED PRACTICAL SQL QUERY

```
CREATE TABLE accounts (  
    acc_no INT PRIMARY KEY,  
    balance DECIMAL(12,2) CHECK (balance >= 0)  
);
```

 EXECUTION OUTPUT RESULT TABLE

Status	Integrity_Level
ACTIVE	Primary Key & Check constraints enforced.

 STEP-BY-STEP LOGIC

1. Evaluates incoming INSERT/UPDATE data against declared rules. 2. Rejects invalid operations.

 PRO TIP & PITFALL

Constraints maintain clean data quality at the database engine level regardless of application code bugs.

NOT NULL Constraint

 CONCEPT NOTES & DEFINITION

Ensures that a column cannot store NULL (missing) values.

 FUNCTION / COMMAND SYNTAX

```
CREATE TABLE table_name (  
    col1 DATATYPE NOT NULL,  
    col2 DATATYPE NOT NULL  
);
```

 SAMPLE INPUT DATA TABLE

Field	Null_Allowed
username	NO
pass_hash	NO

 EXECUTED PRACTICAL SQL QUERY

```
CREATE TABLE users (  
    username VARCHAR(50) NOT NULL,  
    pass_hash VARCHAR(255) NOT NULL  
);
```

 EXECUTION OUTPUT RESULT TABLE

Field	Null_Allowed
username	NO
pass_hash	NO

 STEP-BY-STEP LOGIC

- 1. Declares required status for username and pass_hash columns. 2. Blocks NULL insert attempts.

 PRO TIP & PITFALL

Always declare NOT NULL on mandatory business fields like usernames, passwords, and transaction amounts.

UNIQUE Constraint

 **CONCEPT NOTES & DEFINITION**

Guarantees that all values in a column or set of columns are distinct across all rows.

 FUNCTION / COMMAND SYNTAX

```
CREATE TABLE table_name (  
  col1 INT PRIMARY KEY,  
  col2 VARCHAR(100) UNIQUE  
);
```

 SAMPLE INPUT DATA TABLE

Field	Key_Type
email	UNI (Unique)

 EXECUTED PRACTICAL SQL QUERY

```
CREATE TABLE users (  
  user_id INT PRIMARY KEY,  
  email VARCHAR(100) NOT NULL UNIQUE  
);
```

 EXECUTION OUTPUT RESULT TABLE

Field	Key_Type
email	UNI (Unique)

 **STEP-BY-STEP LOGIC**

1. Builds unique B-tree lookup index. 2. Prevents registration of duplicate email addresses.

 **PRO TIP & PITFALL**

A table can have multiple UNIQUE constraints, whereas it can only have ONE PRIMARY KEY.

PRIMARY KEY Constraint

 **CONCEPT NOTES & DEFINITION**

Uniquely identifies each record in a table by combining NOT NULL and UNIQUE requirements.

 FUNCTION / COMMAND SYNTAX

```
CREATE TABLE table_name (  
    column_name DATATYPE PRIMARY KEY  
);
```

 SAMPLE INPUT DATA TABLE

Field	Key_Type
dept_id	PRI (Primary Key)

 EXECUTED PRACTICAL SQL QUERY

```
CREATE TABLE departments (  
    dept_id INT AUTO_INCREMENT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL  
);
```

 EXECUTION OUTPUT RESULT TABLE

Field	Key_Type
dept_id	PRI (Primary Key)

 **STEP-BY-STEP LOGIC**

1. Designates dept_id as immutable unique record identifier. 2. Automatically creates clustered index.

 **PRO TIP & PITFALL**

Every database table should have a PRIMARY KEY to optimize row lookup speeds and relational joins.

FOREIGN KEY Constraint

★ CONCEPT NOTES & DEFINITION

Establishes a relational link between columns in a child table and a primary key in a parent table.

⚙️ FUNCTION / COMMAND SYNTAX

```
CREATE TABLE child_table (  
    id INT PRIMARY KEY,  
    parent_id INT,  
    FOREIGN KEY (parent_id)  
        REFERENCES parent_table(id)  
);
```

📊 SAMPLE INPUT DATA TABLE

Constraint_Name	Relationship
fk_emp_dept	employees.dept_id -> departments.dept_id

📄 EXECUTED PRACTICAL SQL QUERY

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    dept_id INT NOT NULL,  
    FOREIGN KEY (dept_id)  
        REFERENCES departments(dept_id)  
);
```

📋 EXECUTION OUTPUT RESULT TABLE

Constraint_Name	Relationship
fk_emp_dept	employees.dept_id -> departments.dept_id

💡 STEP-BY-STEP LOGIC

- 1. Validates that dept_id exists in departments parent table before inserting into employees. 2. Rejects orphaned records.

⚠️ PRO TIP & PITFALL

Use ON DELETE CASCADE or ON DELETE SET NULL to manage dependent records when parent rows are deleted.

CHECK Constraint

 CONCEPT NOTES & DEFINITION

Validates that column values satisfy a specified boolean logic condition before allowing insertion.

 FUNCTION / COMMAND SYNTAX

```
CREATE TABLE table_name (  
    id INT PRIMARY KEY,  
    price DECIMAL(8,2) CHECK (price > 0.00)  
);
```

 SAMPLE INPUT DATA TABLE

Status	Validation
ACTIVE	price must be strictly > 0.00

 EXECUTED PRACTICAL SQL QUERY

```
CREATE TABLE products (  
    prod_id INT PRIMARY KEY,  
    price DECIMAL(8,2) CHECK (price > 0.00)  
);
```

 EXECUTION OUTPUT RESULT TABLE

Status	Validation
ACTIVE	price must be strictly > 0.00

 STEP-BY-STEP LOGIC

1. Evaluates price input during INSERT or UPDATE. 2. Raises SQL exception if price <= 0.00.

 PRO TIP & PITFALL

CHECK constraints were fully enforced starting in MySQL 8.0.16.

DEFAULT Constraint

✦ CONCEPT NOTES & DEFINITION

Provides a fallback value for a column when no value is specified during an INSERT statement.

⚙️ FUNCTION / COMMAND SYNTAX

```
CREATE TABLE table_name (  
    id INT PRIMARY KEY,  
    status VARCHAR(20) DEFAULT 'ACTIVE'  
);
```

📊 SAMPLE INPUT DATA TABLE

Column	Default_Value
order_status	'PENDING'
order_date	CURRENT_TIMESTAMP

💻 EXECUTED PRACTICAL SQL QUERY

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    order_status VARCHAR(20) DEFAULT 'PENDING',  
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

📋 EXECUTION OUTPUT RESULT TABLE

Column	Default_Value
order_status	'PENDING'

Column	Default_Value
order_date	CURRENT_TIMESTAMP

💡 STEP-BY-STEP LOGIC

- 1. Inspects INSERT statement.
- 2. Automatically substitutes default values for omitted fields.

🚨 PRO TIP & PITFALL

DEFAULT CURRENT_TIMESTAMP is ideal for audit logging creation times.

AUTO_INCREMENT

★ CONCEPT NOTES & DEFINITION

Automatically generates sequential numbers when new rows are inserted into an integer primary key column.

⚙️ FUNCTION / COMMAND SYNTAX

```
CREATE TABLE table_name (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100)  
);
```

📊 SAMPLE INPUT DATA TABLE

cust_id	cust_name
1001	Acme Corp
1002	Global Tech

📝 EXECUTED PRACTICAL SQL QUERY

```
CREATE TABLE customers (  
    cust_id INT AUTO_INCREMENT PRIMARY KEY,  
    cust_name VARCHAR(100) NOT NULL  
);
```

📄 EXECUTION OUTPUT RESULT TABLE

cust_id	cust_name
1001	Acme Corp
1002	Global Tech

💡 STEP-BY-STEP LOGIC

- 1. Reads current maximum ID counter.
- 2. Increments value by 1 for new row insert.

⚠️ PRO TIP & PITFALL

Never pass manual values into an AUTO_INCREMENT column unless explicitly overriding sequence seed.

CREATE INDEX

 CONCEPT NOTES & DEFINITION

Builds a B-Tree search index on one or more columns to drastically speed up SELECT filter lookups.

 FUNCTION / COMMAND SYNTAX

```
CREATE INDEX index_name
  ON table_name (column_name);
```

 SAMPLE INPUT DATA TABLE

Status	Index_Name
SUCCESS	idx_emp_salary built on employees(salary)

 EXECUTED PRACTICAL SQL QUERY

```
CREATE INDEX idx_emp_salary
  ON employees (salary);
```

 EXECUTION OUTPUT RESULT TABLE

Status	Index_Name
SUCCESS	idx_emp_salary built on employees(salary)

 STEP-BY-STEP LOGIC

- 1. Constructs sorted B-Tree index structure.
- 2. Reduces search complexity from $O(N)$ full table scan to $O(\log N)$.

 PRO TIP & PITFALL

Do not over-index tables with high write volume, as every INSERT/UPDATE requires index recalculation.

DROP INDEX

 CONCEPT NOTES & DEFINITION

Removes an existing B-Tree search index from a table.

 FUNCTION / COMMAND SYNTAX

```
DROP INDEX index_name
ON table_name;
```

 SAMPLE INPUT DATA TABLE

Status	Message
SUCCESS	Index idx_emp_salary dropped.

 EXECUTED PRACTICAL SQL QUERY

```
DROP INDEX idx_emp_salary
ON employees;
```

 EXECUTION OUTPUT RESULT TABLE

Status	Message
SUCCESS	Index idx_emp_salary dropped.

 STEP-BY-STEP LOGIC

1. Deallocates index memory pages.
2. Reverts query engine to standard table access.

 PRO TIP & PITFALL

Remove unused indexes to improve INSERT and UPDATE performance.

MySQL Views

 CONCEPT NOTES & DEFINITION

Creates a virtual table based on the result-set of a pre-defined SQL SELECT query.

 FUNCTION / COMMAND SYNTAX

```
CREATE VIEW view_name AS
SELECT col1, col2
FROM table_name
WHERE condition;
```

 SAMPLE INPUT DATA TABLE

emp_name	salary
Alex Mercer	\$95,000.00
Samantha Ray	\$105,000.00

 EXECUTED PRACTICAL SQL QUERY

```
CREATE VIEW v_high_earners AS
SELECT emp_name, salary
FROM employees
WHERE salary > 90000;
```

 EXECUTION OUTPUT RESULT TABLE

emp_name	salary
Alex Mercer	\$95,000.00
Samantha Ray	\$105,000.00

 STEP-BY-STEP LOGIC

- 1. Encapsulates complex filter query.
- 2. Presents virtual table interface to client applications.

 PRO TIP & PITFALL

Views do not store data physically; they execute the underlying query dynamically when called.

SQL Injection Threats

★ CONCEPT NOTES & DEFINITION

Explains security vulnerabilities where malicious SQL commands are injected into unsafe string inputs.

⚙️ FUNCTION / COMMAND SYNTAX

```
-- UNSAFE CONCATENATED QUERY
SELECT * FROM users
WHERE username = 'admin' AND '1'='1';
```

📊 SAMPLE INPUT DATA TABLE

Status	Vulnerability_Level
EXPOSED	Inline String Concatenation Threat Detected

💻 EXECUTED PRACTICAL SQL QUERY

```
SELECT * FROM users
WHERE username = 'admin' AND '1'='1';
```

📑 EXECUTION OUTPUT RESULT TABLE

Status	Vulnerability_Level
EXPOSED	Inline String Concatenation Threat Detected

💡 STEP-BY-STEP LOGIC

1. Demonstrates how injected boolean logic bypasses authentication. 2. Emphasizes strict sanitization.

⚠️ PRO TIP & PITFALL

NEVER concatenate raw user inputs directly into SQL statements! Always use Prepared Statements.

Prepared Statements

 CONCEPT NOTES & DEFINITION

Pre-compiles SQL execution plans and binds data parameters separately to achieve 100% security against SQL injection.

 FUNCTION / COMMAND SYNTAX

```
PREPARE stmt FROM
    'SELECT * FROM table WHERE col = ?';
SET @val = 101;
EXECUTE stmt USING @val;
```

 SAMPLE INPUT DATA TABLE

emp_name	salary
Alex Mercer	\$95,000.00

 EXECUTED PRACTICAL SQL QUERY

```
PREPARE stmt_user FROM
    'SELECT emp_name, salary FROM employees WHERE emp_id = ?';
SET @eid = 101;
EXECUTE stmt_user USING @eid;
```

 EXECUTION OUTPUT RESULT TABLE

emp_name	salary
Alex Mercer	\$95,000.00

 STEP-BY-STEP LOGIC

- 1. Pre-compiles statement template on MySQL server. 2. Binds parameters safely without string evaluation.

 PRO TIP & PITFALL

Prepared statements also boost execution performance when calling the same query repeatedly with different parameters.

INSERT INTO

 **CONCEPT NOTES & DEFINITION**

Adds new data records into a database table.

 FUNCTION / COMMAND SYNTAX

```
INSERT INTO table_name (  
    col1,  
    col2  
) VALUES (  
    val1,  
    val2  
);
```

 SAMPLE INPUT DATA TABLE

Status	Rows Affected
SUCCESS	1 row inserted.

 EXECUTED PRACTICAL SQL QUERY

```
INSERT INTO employees (  
    emp_name,  
    department,  
    salary  
) VALUES (  
    'John Doe',  
    'Engineering',  
    75000.00  
);
```

 EXECUTION OUTPUT RESULT TABLE

Status	Rows Affected
SUCCESS	1 row inserted.

 **STEP-BY-STEP LOGIC**

1. Validates input data types. 2. Appends new record to table storage.

 **PRO TIP & PITFALL**

Omitted columns will use their DEFAULT values or store NULL if allowed.

SELECT

★ CONCEPT NOTES & DEFINITION

Retrieves specific columns or data expressions from a database table.

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    column1,
    column2
FROM table_name;
```

📊 SAMPLE INPUT DATA TABLE

emp_name	salary
Alex Mercer	\$95,000
Sarah Connor	\$88,000
David Miller	\$92,000

📄 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    salary
FROM employees;
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	salary
Alex Mercer	\$95,000

emp_name	salary
Sarah Connor	\$88,000
David Miller	\$92,000

💡 STEP-BY-STEP LOGIC

1. Scans employees table. 2. Extracts emp_name and salary columns. 3. Displays formatted result set.

⚠️ PRO TIP & PITFALL

Avoid using SELECT * in production code to prevent performance bottlenecks.

SELECT DISTINCT

★ CONCEPT NOTES & DEFINITION

Removes duplicate rows from the query output, returning only unique values.

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT DISTINCT
    column_name
FROM table_name;
```

📊 SAMPLE INPUT DATA TABLE

department
Engineering
Data Science
Product Management

🖨️ EXECUTED PRACTICAL SQL QUERY

```
SELECT DISTINCT
    department
FROM employees;
```

📄 EXECUTION OUTPUT RESULT TABLE

department
Engineering
Data Science
Product Management

💡 STEP-BY-STEP LOGIC

- 1. Reads department column. 2. Identifies duplicates. 3. Filters output to distinct items.

⚠️ PRO TIP & PITFALL

SELECT DISTINCT evaluates all selected columns together.

WHERE

 CONCEPT NOTES & DEFINITION

Filters result rows based on boolean comparison conditions.

 FUNCTION / COMMAND SYNTAX

```
SELECT
    col1,
    col2
FROM table_name
WHERE condition_expression;
```

 SAMPLE INPUT DATA TABLE

emp_name	salary
Alex Mercer	\$95,000
Samantha Ray	\$105,000

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    salary
FROM employees
WHERE salary > 85000.00;
```

 EXECUTION OUTPUT RESULT TABLE

emp_name	salary
Alex Mercer	\$95,000

emp_name	salary
Samantha Ray	\$105,000

 STEP-BY-STEP LOGIC

- 1. Evaluates each row against salary > 85000.
- 2. Keeps rows satisfying condition.

 PRO TIP & PITFALL

Surround string filter values in single quotes
e.g. WHERE dept = 'IT'.

ORDER BY

✦ CONCEPT NOTES & DEFINITION

Sorts query output in ascending (ASC) or descending (DESC) order.

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    col1,
    col2
FROM table_name
ORDER BY col1 ASC|DESC;
```

📊 SAMPLE INPUT DATA TABLE

emp_name	salary
Samantha Ray	\$105,000
Alex Mercer	\$95,000
David Miller	\$92,000

🖨️ EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    salary
FROM employees
ORDER BY salary DESC;
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	salary
Samantha Ray	\$105,000
Alex Mercer	\$95,000
David Miller	\$92,000

💡 STEP-BY-STEP LOGIC

1. Fetches matching records. 2. Sorts salary values in descending order.

⚠️ PRO TIP & PITFALL

Default sorting order is ASC (ascending) if omitted.

AND

 CONCEPT NOTES & DEFINITION

Combines multiple conditions where ALL must be TRUE.

 FUNCTION / COMMAND SYNTAX

```
SELECT
    col1,
    col2
FROM table_name
WHERE cond1 AND cond2;
```

 SAMPLE INPUT DATA TABLE

emp_name	salary
Alex Mercer	\$95,000

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    salary
FROM employees
WHERE department = 'Engineering'
    AND salary >= 90000.00;
```

 EXECUTION OUTPUT RESULT TABLE

emp_name	salary
Alex Mercer	\$95,000

 STEP-BY-STEP LOGIC

- 1. Checks department = Engineering.
- 2. Checks salary >= 90000.
- 3. Returns row if both pass.

 PRO TIP & PITFALL

AND has higher operator precedence than OR.

OR

 **CONCEPT NOTES & DEFINITION**

Combines conditions where AT LEAST ONE must be TRUE.

 FUNCTION / COMMAND SYNTAX

```
SELECT
    col1,
    col2
FROM table_name
WHERE cond1 OR cond2;
```

 SAMPLE INPUT DATA TABLE

emp_name	department
Alex Mercer	Engineering
Samantha Ray	Data Science

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    department
FROM employees
WHERE department = 'Engineering'
    OR department = 'Data Science';
```

 EXECUTION OUTPUT RESULT TABLE

emp_name	department
Alex Mercer	Engineering

emp_name	department
Samantha Ray	Data Science

 **STEP-BY-STEP LOGIC**

1. Checks if row matches either Engineering or Data Science. 2. Emits matching rows.

 **PRO TIP & PITFALL**

Use parentheses () when mixing AND and OR in the same WHERE clause.

NOT

📌 CONCEPT NOTES & DEFINITION

Negates a condition, selecting rows where condition is FALSE.

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    col1,
    col2
FROM table_name
WHERE NOT condition;
```

📊 SAMPLE INPUT DATA TABLE

emp_name	department
Sarah Connor	DevOps
Samantha Ray	Data Science

💻 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    department
FROM employees
WHERE NOT department = 'Engineering';
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	department
Sarah Connor	DevOps

emp_name	department
Samantha Ray	Data Science

💡 STEP-BY-STEP LOGIC

- 1. Evaluates department = Engineering.
- 2. Inverts boolean result to select non-Engineering staff.

🚨 PRO TIP & PITFALL

Equivalent to using the not equal operator (<> or !=).

NULL Values

 CONCEPT NOTES & DEFINITION

Checks for missing data using IS NULL or IS NOT NULL.

 FUNCTION / COMMAND SYNTAX

```
SELECT
    col1,
    col2
FROM table_name
WHERE column IS [NOT] NULL;
```

 SAMPLE INPUT DATA TABLE

emp_name	bonus
Alex Mercer	\$5,000
Samantha Ray	\$10,000

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    bonus
FROM employees
WHERE bonus IS NOT NULL;
```

 EXECUTION OUTPUT RESULT TABLE

emp_name	bonus
Alex Mercer	\$5,000

emp_name	bonus
Samantha Ray	\$10,000

 STEP-BY-STEP LOGIC

- 1. Inspects bonus column. 2. Filters out records containing NULL values.

 PRO TIP & PITFALL

Never use = NULL or != NULL. Always use IS NULL or IS NOT NULL.

UPDATE

 **CONCEPT NOTES & DEFINITION**

Modifies existing column data in a table.

 FUNCTION / COMMAND SYNTAX

```
UPDATE table_name
SET
    col1 = val1,
    col2 = val2
WHERE condition_expression;
```

 SAMPLE INPUT DATA TABLE

Status	Rows Updated
SUCCESS	1 row updated.

 EXECUTED PRACTICAL SQL QUERY

```
UPDATE employees
SET
    salary = 98000.00
WHERE emp_id = 101;
```

 EXECUTION OUTPUT RESULT TABLE

Status	Rows Updated
SUCCESS	1 row updated.

 **STEP-BY-STEP LOGIC**

 **PRO TIP & PITFALL**

1. Finds row where emp_id = 101. 2. Sets new salary value to 98000.00.

ALWAYS include a WHERE clause in UPDATE statements to avoid modifying all table rows!

DELETE

 CONCEPT NOTES & DEFINITION

Removes records from a table based on a condition.

 FUNCTION / COMMAND SYNTAX

```
DELETE FROM table_name
WHERE condition_expression;
```

 SAMPLE INPUT DATA TABLE

Status	Rows Deleted
SUCCESS	3 rows deleted.

 EXECUTED PRACTICAL SQL QUERY

```
DELETE FROM employees
WHERE status = 'Terminated';
```

 EXECUTION OUTPUT RESULT TABLE

Status	Rows Deleted
SUCCESS	3 rows deleted.

 STEP-BY-STEP LOGIC

- 1. Identifies rows matching status = 'Terminated'. 2. Deletes matching records.

 PRO TIP & PITFALL

Omit WHERE clause only if you intend to delete ALL rows in the table.

LIMIT

★
CONCEPT NOTES & DEFINITION

Restricts row count for pagination

⚙️
FUNCTION / COMMAND SYNTAX

```

SELECT
    col1,
    col2
FROM table_name
LIMIT count OFFSET offset;

```

📊
SAMPLE INPUT DATA TABLE

emp_name	salary
Samantha Ray	\$105,000
Alex Mercer	\$95,000
David Miller	\$92,000

📄
EXECUTED PRACTICAL SQL QUERY

```

SELECT
    emp_name,
    salary
FROM employees
ORDER BY salary DESC
LIMIT 3;

```

📄
EXECUTION OUTPUT RESULT TABLE

emp_name	salary
Samantha Ray	\$105,000
Alex Mercer	\$95,000
David Miller	\$92,000

💡
STEP-BY-STEP LOGIC

1. Sorts employees by salary. 2. Returns top 3 rows.

⚠️
PRO TIP & PITFALL

OFFSET allows skipping N initial rows for pagination.

Aggregate Functions

 CONCEPT NOTES & DEFINITION

Summary statistics calculation

 FUNCTION / COMMAND SYNTAX

```
SELECT
    COUNT(*) AS total_rows,
    AVG(col) AS avg_value
FROM table_name;
```

 SAMPLE INPUT DATA TABLE

total_staff	avg_salary
150	\$75,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    COUNT(*) AS total_staff,
    AVG(salary) AS avg_salary
FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

total_staff	avg_salary
150	\$75,000.00

 STEP-BY-STEP LOGIC

1. Computes total row count and average salary.

 PRO TIP & PITFALL

Aggregate functions ignore NULL values except COUNT(*).

MIN()

📌 CONCEPT NOTES & DEFINITION

Returns lowest column value

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    MIN(column_name) AS min_val
FROM table_name;
```

📊 SAMPLE INPUT DATA TABLE

lowest_salary
\$45,000.00

🖨️ EXECUTED PRACTICAL SQL QUERY

```
SELECT
    MIN(salary) AS lowest_salary
FROM employees;
```

📋 EXECUTION OUTPUT RESULT TABLE

lowest_salary
\$45,000.00

💡 STEP-BY-STEP LOGIC

1. Scans salary column. 2. Finds minimum value.

⚠️ PRO TIP & PITFALL

Works on numbers, strings, and dates.

MAX()

 CONCEPT NOTES & DEFINITION

Returns highest column value

 FUNCTION / COMMAND SYNTAX

```
SELECT
    MAX(column_name) AS max_val
FROM table_name;
```

 SAMPLE INPUT DATA TABLE

highest_salary
\$165,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    MAX(salary) AS highest_salary
FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

highest_salary
\$165,000.00

 STEP-BY-STEP LOGIC

1. Scans salary column. 2. Finds maximum value.

 PRO TIP & PITFALL

Works on numbers, strings, and dates.

COUNT()

 CONCEPT NOTES & DEFINITION

Counts total matching rows

 FUNCTION / COMMAND SYNTAX

```
SELECT
    COUNT(column_name) AS total_count
FROM table_name;
```

 SAMPLE INPUT DATA TABLE

total_staff
150

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    COUNT(*) AS total_staff
FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

total_staff
150

 STEP-BY-STEP LOGIC

- 1. Counts total records in result set.

 PRO TIP & PITFALL

COUNT(column) ignores NULLs; COUNT(*) includes NULLs.

SUM()

 CONCEPT NOTES & DEFINITION

Calculates total numerical addition

 FUNCTION / COMMAND SYNTAX

```
SELECT
    SUM(column_name) AS total_sum
FROM table_name;
```

 SAMPLE INPUT DATA TABLE

total_payroll
\$11,250,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    SUM(salary) AS total_payroll
FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

total_payroll
\$11,250,000.00

 STEP-BY-STEP LOGIC

1. Adds all numeric column values together.

 PRO TIP & PITFALL

Returns NULL if no matching rows exist.

AVG()

 CONCEPT NOTES & DEFINITION

Calculates arithmetic mean average

 FUNCTION / COMMAND SYNTAX

```
SELECT
    AVG(column_name) AS average_val
FROM table_name;
```

 SAMPLE INPUT DATA TABLE

average_salary

\$75,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    AVG(salary) AS average_salary
FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

average_salary

\$75,000.00

 STEP-BY-STEP LOGIC

1. Divides total sum by non-null row count.

 PRO TIP & PITFALL

Ignores NULL entries in calculation.

LIKE

★ CONCEPT NOTES & DEFINITION

Pattern matching with wildcards

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    col1,
    col2
FROM table_name
WHERE col1 LIKE 'pattern';
```

📊 SAMPLE INPUT DATA TABLE

emp_name	department
Alex Mercer	Engineering
Alice Walker	HR

🖨️ EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    department
FROM employees
WHERE emp_name LIKE 'A%';
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	department
Alex Mercer	Engineering

emp_name	department
Alice Walker	HR

💡 STEP-BY-STEP LOGIC

- 1. Matches names starting with letter 'A'.

⚠️ PRO TIP & PITFALL

Case-insensitive depending on collation.

Wildcards

 **CONCEPT NOTES & DEFINITION**

Using % and _ in pattern queries

 **FUNCTION / COMMAND SYNTAX**

```
SELECT
    col1
FROM table_name
WHERE col1 LIKE '_a%';
```

 **SAMPLE INPUT DATA TABLE**

emp_name
Sarah Connor
David Miller

 **EXECUTED PRACTICAL SQL QUERY**

```
SELECT
    emp_name
FROM employees
WHERE emp_name LIKE '_a%';
```

 **EXECUTION OUTPUT RESULT TABLE**

emp_name
Sarah Connor
David Miller

 **STEP-BY-STEP LOGIC**

1. Matches strings with 'a' as second letter.

 **PRO TIP & PITFALL**

% matches zero or more chars; _ matches 1 char.

IN

★ CONCEPT NOTES & DEFINITION

Matches value against candidate list

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    col1,
    col2
FROM table_name
WHERE col1 IN (val1, val2);
```

📊 SAMPLE INPUT DATA TABLE

emp_name	department
Alex Mercer	Engineering
Sarah Connor	DevOps

📄 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    department
FROM employees
WHERE department IN ('Engineering', 'DevOps');
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	department
Alex Mercer	Engineering

emp_name	department
Sarah Connor	DevOps

💡 STEP-BY-STEP LOGIC

- 1. Checks if department equals any item in list.

⚠️ PRO TIP & PITFALL

Shorthand for multiple OR conditions.

BETWEEN

★ CONCEPT NOTES & DEFINITION

Inclusive minimum and maximum range filter

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    col1,
    col2
FROM table_name
WHERE col1 BETWEEN min AND max;
```

📊 SAMPLE INPUT DATA TABLE

emp_name	salary
Alex Mercer	\$95,000
Sarah Connor	\$88,000

🖨️ EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    salary
FROM employees
WHERE salary BETWEEN 80000.00 AND 100000.00;
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	salary
Alex Mercer	\$95,000

emp_name	salary
Sarah Connor	\$88,000

💡 STEP-BY-STEP LOGIC

1. Filters salaries between \$80k and \$100k.

⚠️ PRO TIP & PITFALL

Includes boundary values min and max.

Aliases

 **CONCEPT NOTES & DEFINITION**

Renames output column or table names

 FUNCTION / COMMAND SYNTAX

```
SELECT
    col1 AS alias1,
    col2 AS alias2
FROM table_name AS t_alias;
```

 SAMPLE INPUT DATA TABLE

EmployeeName	MonthlySalary
Alex Mercer	\$95,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name AS EmployeeName,
    salary AS MonthlySalary
FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

EmployeeName	MonthlySalary
Alex Mercer	\$95,000.00

 **STEP-BY-STEP LOGIC**

1. Renames output header columns.

 **PRO TIP & PITFALL**

Aliases exist only for query execution duration.

Joins

★ CONCEPT NOTES & DEFINITION

Combines rows from related tables

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    t1.col1,
    t2.col2
FROM table1 t1
JOIN table2 t2
    ON t1.id = t2.t1_id;
```

📊 SAMPLE INPUT DATA TABLE

emp_name	dept_name
Alex Mercer	Engineering

💻 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    e.emp_name,
    d.dept_name
FROM employees e
JOIN departments d
    ON e.dept_id = d.id;
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	dept_name
Alex Mercer	Engineering

💡 STEP-BY-STEP LOGIC

1. Links employees and departments tables.

⚠️ PRO TIP & PITFALL

Must specify ON clause condition.

INNER JOIN

★ CONCEPT NOTES & DEFINITION

Matching rows in both tables

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    t1.col1,
    t2.col2
FROM table1 t1
INNER JOIN table2 t2
    ON t1.id = t2.t1_id;
```

📊 SAMPLE INPUT DATA TABLE

emp_name	dept_name
Alex Mercer	Engineering

💻 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    e.emp_name,
    d.dept_name
FROM employees e
INNER JOIN departments d
    ON e.dept_id = d.id;
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	dept_name
Alex Mercer	Engineering

💡 STEP-BY-STEP LOGIC

1. Emits rows where foreign key matches.

⚠️ PRO TIP & PITFALL

Unmatched rows in either table are excluded.

LEFT JOIN

★ CONCEPT NOTES & DEFINITION

All left table rows + matched right

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    t1.col1,
    t2.col2
FROM table1 t1
LEFT JOIN table2 t2
    ON t1.id = t2.t1_id;
```

📊 SAMPLE INPUT DATA TABLE

emp_name	dept_name
Alex Mercer	Engineering
John Unassigned	NULL

📄 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    e.emp_name,
    d.dept_name
FROM employees e
LEFT JOIN departments d
    ON e.dept_id = d.id;
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	dept_name
Alex Mercer	Engineering
John Unassigned	NULL

💡 STEP-BY-STEP LOGIC

1.Returns all left rows regardless of match.

⚠️ PRO TIP & PITFALL

Unmatched right columns return NULL.

RIGHT JOIN

★ CONCEPT NOTES & DEFINITION

All right table rows + matched left

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
    t1.col1,
    t2.col2
FROM table1 t1
RIGHT JOIN table2 t2
    ON t1.id = t2.t1_id;
```

📊 SAMPLE INPUT DATA TABLE

emp_name	dept_name
Alex Mercer	Engineering
NULL	Empty Dept

📄 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    e.emp_name,
    d.dept_name
FROM employees e
RIGHT JOIN departments d
    ON e.dept_id = d.id;
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	dept_name
Alex Mercer	Engineering
NULL	Empty Dept

💡 STEP-BY-STEP LOGIC

1.Returns all right rows regardless of match.

⚠️ PRO TIP & PITFALL

Unmatched left columns return NULL.

CROSS JOIN

 CONCEPT NOTES & DEFINITION

Cartesian product of all rows

 FUNCTION / COMMAND SYNTAX

```
SELECT
  t1.col1,
  t2.col2
FROM table1 t1
CROSS JOIN table2 t2;
```

 SAMPLE INPUT DATA TABLE

emp_name	dept_name
Alex Mercer	Engineering
Alex Mercer	DevOps

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  e.emp_name,
  d.dept_name
FROM employees e
CROSS JOIN departments d;
```

 EXECUTION OUTPUT RESULT TABLE

emp_name	dept_name
Alex Mercer	Engineering

emp_name	dept_name
Alex Mercer	DevOps

 STEP-BY-STEP LOGIC

- 1. Combines every left row with every right row.

 PRO TIP & PITFALL

Generates N * M output rows.

Self Join

★ CONCEPT NOTES & DEFINITION

Joins table to itself

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
  a.col1 AS Worker,
  b.col1 AS Manager
FROM table1 a
LEFT JOIN table1 b
  ON a.mgr_id = b.id;
```

📊 SAMPLE INPUT DATA TABLE

Worker	Manager
Alex Mercer	Sarah Connor

💻 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  e.emp_name AS Worker,
  m.emp_name AS Manager
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.id;
```

📄 EXECUTION OUTPUT RESULT TABLE

Worker	Manager
Alex Mercer	Sarah Connor

💡 STEP-BY-STEP LOGIC

1. Treats single table as two separate aliases.

⚠️ PRO TIP & PITFALL

Requires distinct table aliases.

UNION

 **CONCEPT NOTES & DEFINITION**

Combines distinct query results

 FUNCTION / COMMAND SYNTAX

```
SELECT col1 FROM table1
UNION
SELECT col1 FROM table2;
```

 SAMPLE INPUT DATA TABLE

city
London
Paris

 EXECUTED PRACTICAL SQL QUERY

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;
```

 EXECUTION OUTPUT RESULT TABLE

city
London
Paris

 **STEP-BY-STEP LOGIC**

 **PRO TIP & PITFALL**

1. Merges result sets. 2. Removes duplicates.

Queries must return same column count and types.

UNION ALL

★ **CONCEPT NOTES & DEFINITION**
Combines query results with duplicates

⚙ **FUNCTION / COMMAND SYNTAX**

```
SELECT col1 FROM table1
UNION ALL
SELECT col1 FROM table2;
```

📊 **SAMPLE INPUT DATA TABLE**

city
London
London
Paris

📄 **EXECUTED PRACTICAL SQL QUERY**

```
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;
```

📄 **EXECUTION OUTPUT RESULT TABLE**

city
London
London
Paris

💡 **STEP-BY-STEP LOGIC**
1. Merges result sets retaining all rows.

⚠ **PRO TIP & PITFALL**
Faster than UNION because duplicate check is skipped.

GROUP BY

 CONCEPT NOTES & DEFINITION

Groups rows for summary calculations

 FUNCTION / COMMAND SYNTAX

```
SELECT
  department,
  COUNT(*) AS emp_count
FROM employees
GROUP BY department;
```

 SAMPLE INPUT DATA TABLE

department	emp_count
Engineering	4
DevOps	2

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  department,
  COUNT(*) AS emp_count
FROM employees
GROUP BY department;
```

 EXECUTION OUTPUT RESULT TABLE

department	emp_count
Engineering	4

department	emp_count
DevOps	2

 STEP-BY-STEP LOGIC

- 1. Buckets rows by department. 2. Counts items.

 PRO TIP & PITFALL

Non-aggregated SELECT columns must be in GROUP BY.

HAVING

 CONCEPT NOTES & DEFINITION

Filters aggregate group results

 FUNCTION / COMMAND SYNTAX

```
SELECT
  department,
  SUM(salary) AS total_sal
FROM employees
GROUP BY department
HAVING SUM(salary) > 300000.00;
```

 SAMPLE INPUT DATA TABLE

department	total_payroll
Engineering	\$380,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  department,
  SUM(salary) AS total_payroll
FROM employees
GROUP BY department
HAVING SUM(salary) > 300000.00;
```

 EXECUTION OUTPUT RESULT TABLE

department	total_payroll
Engineering	\$380,000.00

 STEP-BY-STEP LOGIC

- 1. Calculates department sums. 2. Filters totals > \$300k.

 PRO TIP & PITFALL

WHERE filters rows before grouping; HAVING filters after.

EXISTS

 CONCEPT NOTES & DEFINITION

Tests subquery row presence

 FUNCTION / COMMAND SYNTAX

```
SELECT
    dept_name
FROM departments d
WHERE EXISTS (
    SELECT 1 FROM employees e
    WHERE e.dept_id = d.id
);
```

 SAMPLE INPUT DATA TABLE

dept_name
Engineering

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    dept_name
FROM departments d
WHERE EXISTS (
    SELECT 1 FROM employees e
    WHERE e.dept_id = d.id
);
```

 EXECUTION OUTPUT RESULT TABLE

dept_name
Engineering

 STEP-BY-STEP LOGIC

- 1. Returns TRUE if subquery yields 1+ rows.

 PRO TIP & PITFALL

More efficient than IN for large subqueries.

ANY

★ **CONCEPT NOTES & DEFINITION**

Matches ANY subquery value

⚙️ **FUNCTION / COMMAND SYNTAX**

```
SELECT
    emp_name
FROM employees
WHERE salary > ANY (
    SELECT salary FROM employees
    WHERE department = 'DevOps'
);
```

📊 **SAMPLE INPUT DATA TABLE**

emp_name
Alex Mercer

🖨️ **EXECUTED PRACTICAL SQL QUERY**

```
SELECT
    emp_name
FROM employees
WHERE salary > ANY (
    SELECT salary FROM employees
    WHERE department = 'DevOps'
);
```

📋 **EXECUTION OUTPUT RESULT TABLE**

emp_name
Alex Mercer

💡 **STEP-BY-STEP LOGIC**

1. Returns TRUE if value > minimum subquery value.

⚠️ **PRO TIP & PITFALL**

ANY is equivalent to SOME.

ALL

★ **CONCEPT NOTES & DEFINITION**

Matches ALL subquery values

⚙️ **FUNCTION / COMMAND SYNTAX**

```
SELECT
    emp_name
FROM employees
WHERE salary > ALL (
    SELECT salary FROM employees
    WHERE department = 'DevOps'
);
```

📊 **SAMPLE INPUT DATA TABLE**

emp_name
Samantha Ray

📄 **EXECUTED PRACTICAL SQL QUERY**

```
SELECT
    emp_name
FROM employees
WHERE salary > ALL (
    SELECT salary FROM employees
    WHERE department = 'DevOps'
);
```

📄 **EXECUTION OUTPUT RESULT TABLE**

emp_name
Samantha Ray

💡 **STEP-BY-STEP LOGIC**

- 1. Returns TRUE if value > maximum subquery value.

⚠️ **PRO TIP & PITFALL**

Fails if subquery returns no rows.

INSERT SELECT

★ CONCEPT NOTES & DEFINITION

Copies rows from query into another table

⚙️ FUNCTION / COMMAND SYNTAX

```
INSERT INTO archived_employees (  
    emp_id,  
    emp_name,  
    salary  
)  
SELECT  
    emp_id,  
    emp_name,  
    salary  
FROM employees  
WHERE status = 'Inactive';
```

📊 SAMPLE INPUT DATA TABLE

Status
12 rows archived.

📋 EXECUTED PRACTICAL SQL QUERY

```
INSERT INTO archived_employees (  
    emp_id,  
    emp_name,  
    salary  
)  
SELECT  
    emp_id,  
  
    emp_name,  
    salary  
FROM employees  
WHERE status = 'Inactive';
```

📄 EXECUTION OUTPUT RESULT TABLE

Status
12 rows archived.

💡 STEP-BY-STEP LOGIC

- 1. Queries inactive staff. 2. Inserts into archive.

⚠️ PRO TIP & PITFALL

Column count and types must match target table.

CASE

✦ CONCEPT NOTES & DEFINITION

Multi-branch conditional logic

⚙️ FUNCTION / COMMAND SYNTAX

```
SELECT
  emp_name,
  CASE
    WHEN salary >= 90000 THEN 'Executive'
    ELSE 'Standard'
  END AS Grade
FROM employees;
```

📊 SAMPLE INPUT DATA TABLE

emp_name	Grade
Alex Mercer	Executive

🖨️ EXECUTED PRACTICAL SQL QUERY

```
SELECT
  emp_name,
  CASE
    WHEN salary >= 90000 THEN 'Executive'
    ELSE 'Standard'
  END AS Grade
FROM employees;
```

📄 EXECUTION OUTPUT RESULT TABLE

emp_name	Grade
Alex Mercer	Executive

💡 STEP-BY-STEP LOGIC

1. Evaluates salary >= 90000. 2. Emits Grade.

⚠️ PRO TIP & PITFALL

Always include END keyword.

Null Functions

 CONCEPT NOTES & DEFINITION

Handles NULL values gracefully

 FUNCTION / COMMAND SYNTAX

```
SELECT
    emp_name,
    IFNULL(bonus, 0) AS safe_bonus
FROM employees;
```

 SAMPLE INPUT DATA TABLE

emp_name	safe_bonus
Alex Mercer	\$5,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    emp_name,
    IFNULL(bonus, 0) AS safe_bonus
FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

emp_name	safe_bonus
Alex Mercer	\$5,000.00

 STEP-BY-STEP LOGIC

1. Replaces NULL bonus with 0.

 PRO TIP & PITFALL

Prevents NULL propagation in math.

Stored Procedures

 **CONCEPT NOTES & DEFINITION**

Saved compiled server SQL routines

 FUNCTION / COMMAND SYNTAX

```
CREATE PROCEDURE GetDepartmentPayroll(  
  IN dept_name VARCHAR(50)  
)  
BEGIN  
  SELECT SUM(salary)  
  FROM employees  
  WHERE department = dept_name;  
END;
```

 SAMPLE INPUT DATA TABLE

Status
Procedure executed.

 EXECUTED PRACTICAL SQL QUERY

```
CALL GetDepartmentPayroll('Engineering');
```

 EXECUTION OUTPUT RESULT TABLE

Status
Procedure executed.

 STEP-BY-STEP LOGIC

1. Invokes pre-compiled routine on server.

 PRO TIP & PITFALL

Reduces network bandwidth for complex queries.

Comments & Operators

 CONCEPT NOTES & DEFINITION

SQL documentation and math operators

 FUNCTION / COMMAND SYNTAX

```
-- SINGLE LINE COMMENT
/* MULTI-LINE COMMENT */
SELECT
    10 + 5 AS SumVal,
    10 * 2 AS MultVal;
```

 SAMPLE INPUT DATA TABLE

SumVal	MultVal
15	20

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    10 + 5 AS SumVal,
    10 * 2 AS MultVal;
```

 EXECUTION OUTPUT RESULT TABLE

SumVal	MultVal
15	20

 STEP-BY-STEP LOGIC

1. Executes arithmetic addition and multiplication.

 PRO TIP & PITFALL

Use -- for single line comments, /* */ for multi-line.

ASCII

 CONCEPT NOTES & DEFINITION

ASCII code of first char

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ASCII(str);
```

 SAMPLE INPUT DATA TABLE

ASCII_Code
84

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ASCII('TechBridge') AS ASCII_Code;
```

 EXECUTION OUTPUT RESULT TABLE

ASCII_Code
84

 STEP-BY-STEP LOGIC

1. Evaluates SQL function ASCII. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CHAR_LENGTH

 CONCEPT NOTES & DEFINITION

Character count of string

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CHAR_LENGTH(str);
```

 SAMPLE INPUT DATA TABLE

CharCount
10

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CHAR_LENGTH('TechBridge') AS CharCount;
```

 EXECUTION OUTPUT RESULT TABLE

CharCount
10

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CHAR_LENGTH. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CHARACTER_LENGTH

 CONCEPT NOTES & DEFINITION

Synonym for CHAR_LENGTH

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CHARACTER_LENGTH(str);
```

 SAMPLE INPUT DATA TABLE

CharCount
5

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CHARACTER_LENGTH('MySQL') AS CharCount;
```

 EXECUTION OUTPUT RESULT TABLE

CharCount
5

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CHARACTER_LENGTH. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CONCAT

 CONCEPT NOTES & DEFINITION

Glues string expressions

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CONCAT(s1, s2);
```

 SAMPLE INPUT DATA TABLE

ConcatResult
TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CONCAT('Tech', 'Bridge') AS ConcatResult;
```

 EXECUTION OUTPUT RESULT TABLE

ConcatResult
TechBridge

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function CONCAT.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CONCAT_WS

 CONCEPT NOTES & DEFINITION

Concatenates with separator

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CONCAT_WS(sep, s1, s2);
```

 SAMPLE INPUT DATA TABLE

FormattedDate
2026-09-28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CONCAT_WS('-', '2026', '09', '28') AS FormattedDate;
```

 EXECUTION OUTPUT RESULT TABLE

FormattedDate
2026-09-28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CONCAT_WS. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

FIELD

 CONCEPT NOTES & DEFINITION

Index position in list

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT FIELD(val, s1, s2);
```

 SAMPLE INPUT DATA TABLE

MatchPos
2

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  FIELD('Bridge', 'Tech', 'Bridge') AS MatchPos;
```

 EXECUTION OUTPUT RESULT TABLE

MatchPos
2

 STEP-BY-STEP LOGIC

1. Evaluates SQL function FIELD. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

FIND_IN_SET

 CONCEPT NOTES & DEFINITION

Position in comma list

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT FIND_IN_SET(str, set);
```

 SAMPLE INPUT DATA TABLE

SetPos
3

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  FIND_IN_SET('SQL', 'HTML,CSS,SQL') AS SetPos;
```

 EXECUTION OUTPUT RESULT TABLE

SetPos
3

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function FIND_IN_SET.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

FORMAT

 CONCEPT NOTES & DEFINITION

Formats number with commas

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT FORMAT(X, D);
```

 SAMPLE INPUT DATA TABLE

FormattedNum
1,234,567.89

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  FORMAT(1234567.891, 2) AS FormattedNum;
```

 EXECUTION OUTPUT RESULT TABLE

FormattedNum
1,234,567.89

 STEP-BY-STEP LOGIC

1. Evaluates SQL function FORMAT. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

INSERT

 CONCEPT NOTES & DEFINITION

Replaces string segment

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT INSERT(str, pos, len, new);
```

 SAMPLE INPUT DATA TABLE

InsertedResult
TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  INSERT('TechData', 5, 4, 'Bridge') AS InsertedResult;
```

 EXECUTION OUTPUT RESULT TABLE

InsertedResult
TechBridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function INSERT. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

INSTR

 CONCEPT NOTES & DEFINITION

First substring position

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT INSTR(str, substr);
```

 SAMPLE INPUT DATA TABLE

MatchPos
5

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  INSTR('TechBridge', 'Bridge') AS MatchPos;
```

 EXECUTION OUTPUT RESULT TABLE

MatchPos
5

 STEP-BY-STEP LOGIC

1. Evaluates SQL function INSTR. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LCASE

 CONCEPT NOTES & DEFINITION

Converts to lowercase

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LCASE(str);
```

 SAMPLE INPUT DATA TABLE

LowerResult
techbridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LCASE('TECHBRIDGE') AS LowerResult;
```

 EXECUTION OUTPUT RESULT TABLE

LowerResult
techbridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LCASE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LEFT

 CONCEPT NOTES & DEFINITION

Extracts N left chars

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LEFT(str, len);
```

 SAMPLE INPUT DATA TABLE

LeftPrefix
Tech

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LEFT('TechBridge', 4) AS LeftPrefix;
```

 EXECUTION OUTPUT RESULT TABLE

LeftPrefix
Tech

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LEFT. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LENGTH

 CONCEPT NOTES & DEFINITION

Byte length of string

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LENGTH(str);
```

 SAMPLE INPUT DATA TABLE

ByteLength
10

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LENGTH('TechBridge') AS ByteLength;
```

 EXECUTION OUTPUT RESULT TABLE

ByteLength
10

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function LENGTH.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LOCATE

 CONCEPT NOTES & DEFINITION

Finds index of substring

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LOCATE(substr, str);
```

 SAMPLE INPUT DATA TABLE

FoundPos
5

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LOCATE('Bridge', 'TechBridge') AS FoundPos;
```

 EXECUTION OUTPUT RESULT TABLE

FoundPos
5

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LOCATE. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

LOWER

 CONCEPT NOTES & DEFINITION

Converts to lowercase

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LOWER(str);
```

 SAMPLE INPUT DATA TABLE

LowerResult
techbridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LOWER('TechBridge') AS LowerResult;
```

 EXECUTION OUTPUT RESULT TABLE

LowerResult
techbridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LOWER. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LPAD

 CONCEPT NOTES & DEFINITION

Left-pads string

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LPAD(str, len, pad);
```

 SAMPLE INPUT DATA TABLE

PaddedLeft
000101

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LPAD('101', 6, '0') AS PaddedLeft;
```

 EXECUTION OUTPUT RESULT TABLE

PaddedLeft
000101

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LPAD. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LTRIM

 CONCEPT NOTES & DEFINITION

Strips leading spaces

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LTRIM(str);
```

 SAMPLE INPUT DATA TABLE

TrimmedLeading
TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LTRIM('  TechBridge') AS TrimmedLeading;
```

 EXECUTION OUTPUT RESULT TABLE

TrimmedLeading
TechBridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LTRIM. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MID

 CONCEPT NOTES & DEFINITION

Extracts middle substring

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MID(str, pos, len);
```

 SAMPLE INPUT DATA TABLE

MidSegment
Bridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  MID('TechBridge', 5, 6) AS MidSegment;
```

 EXECUTION OUTPUT RESULT TABLE

MidSegment
Bridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function MID. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

POSITION

 CONCEPT NOTES & DEFINITION

ANSI search position

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT POSITION(sub IN str);
```

 SAMPLE INPUT DATA TABLE

FoundPos
5

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  POSITION('Bridge' IN 'TechBridge') AS FoundPos;
```

 EXECUTION OUTPUT RESULT TABLE

FoundPos
5

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function POSITION.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

REPEAT

 CONCEPT NOTES & DEFINITION

Repeats string N times

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT REPEAT(str, count);
```

 SAMPLE INPUT DATA TABLE

RepeatedText
TechTechTech

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  REPEAT('Tech', 3) AS RepeatedText;
```

 EXECUTION OUTPUT RESULT TABLE

RepeatedText
TechTechTech

 STEP-BY-STEP LOGIC

1. Evaluates SQL function REPEAT. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

REPLACE

 CONCEPT NOTES & DEFINITION

Replaces substring matches

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT REPLACE(str, from, to);
```

 SAMPLE INPUT DATA TABLE

ReplacedResult
TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  REPLACE('W3', 'W3', 'TechBridge') AS ReplacedResult;
```

 EXECUTION OUTPUT RESULT TABLE

ReplacedResult
TechBridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function REPLACE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

REVERSE

 CONCEPT NOTES & DEFINITION

Reverses string characters

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT REVERSE(str);
```

 SAMPLE INPUT DATA TABLE

ReversedText
egdirBhcetT

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  REVERSE('TechBridge') AS ReversedText;
```

 EXECUTION OUTPUT RESULT TABLE

ReversedText
egdirBhcetT

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function REVERSE.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

RIGHT

 CONCEPT NOTES & DEFINITION

Extracts N right chars

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT RIGHT(str, len);
```

 SAMPLE INPUT DATA TABLE

RightSuffix
Bridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  RIGHT('TechBridge', 6) AS RightSuffix;
```

 EXECUTION OUTPUT RESULT TABLE

RightSuffix
Bridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function RIGHT. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

RPAD

 CONCEPT NOTES & DEFINITION

Right-pads string

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT RPAD(str, len, pad);
```

 SAMPLE INPUT DATA TABLE

PaddedRight
Tech****

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  RPAD('Tech', 8, '*') AS PaddedRight;
```

 EXECUTION OUTPUT RESULT TABLE

PaddedRight
Tech****

 STEP-BY-STEP LOGIC

1. Evaluates SQL function RPAD. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

RTRIM

 CONCEPT NOTES & DEFINITION

Strips trailing spaces

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT RTRIM(str);
```

 SAMPLE INPUT DATA TABLE

TrimmedTrailing
TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  RTRIM('TechBridge  ') AS TrimmedTrailing;
```

 EXECUTION OUTPUT RESULT TABLE

TrimmedTrailing
TechBridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function RTRIM. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SPACE

 CONCEPT NOTES & DEFINITION

Generates N spaces

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SPACE(N);
```

 SAMPLE INPUT DATA TABLE

SpacedResult
Tech Bridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CONCAT('Tech', SPACE(4), 'Bridge') AS SpacedResult;
```

 EXECUTION OUTPUT RESULT TABLE

SpacedResult
Tech Bridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SPACE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

STRCMP

 CONCEPT NOTES & DEFINITION

Compares two strings

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT STRCMP(s1, s2);
```

 SAMPLE INPUT DATA TABLE

CompareResult
0

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  STRCMP('Tech', 'Tech') AS CompareResult;
```

 EXECUTION OUTPUT RESULT TABLE

CompareResult
0

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function STRCMP.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SUBSTR

 CONCEPT NOTES & DEFINITION

Extracts substring

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SUBSTR(str, pos, len);
```

 SAMPLE INPUT DATA TABLE

SubResult
Bridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SUBSTR('TechBridge', 5, 6) AS SubResult;
```

 EXECUTION OUTPUT RESULT TABLE

SubResult
Bridge

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function SUBSTR.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SUBSTRING

 CONCEPT NOTES & DEFINITION

Extracts substring

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SUBSTRING(str, pos, len);
```

 SAMPLE INPUT DATA TABLE

SubResult
Tech

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SUBSTRING('TechBridge', 1, 4) AS SubResult;
```

 EXECUTION OUTPUT RESULT TABLE

SubResult
Tech

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function SUBSTRING.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SUBSTRING_INDEX

 CONCEPT NOTES & DEFINITION

Substring before Nth delim

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SUBSTRING_INDEX(str, delim, cnt);
```

 SAMPLE INPUT DATA TABLE

SubIndexResult
a.b

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SUBSTRING_INDEX('a.b.c', '.', 2) AS SubIndexResult;
```

 EXECUTION OUTPUT RESULT TABLE

SubIndexResult
a.b

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SUBSTRING_INDEX.
2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

TRIM

 CONCEPT NOTES & DEFINITION

Removes leading/trailing spaces

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT TRIM(str);
```

 SAMPLE INPUT DATA TABLE

TrimmedBoth
TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  TRIM('  TechBridge  ') AS TrimmedBoth;
```

 EXECUTION OUTPUT RESULT TABLE

TrimmedBoth
TechBridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function TRIM. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

UCASE

 CONCEPT NOTES & DEFINITION

Converts to uppercase

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT UCASE(str);
```

 SAMPLE INPUT DATA TABLE

UpperResult
TECHBRIDGE

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    UCASE('techbridge') AS UpperResult;
```

 EXECUTION OUTPUT RESULT TABLE

UpperResult
TECHBRIDGE

 STEP-BY-STEP LOGIC

1. Evaluates SQL function UCASE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

UPPER

 CONCEPT NOTES & DEFINITION

Converts to uppercase

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT UPPER(str);
```

 SAMPLE INPUT DATA TABLE

UpperResult
TECHBRIDGE

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  UPPER('techbridge') AS UpperResult;
```

 EXECUTION OUTPUT RESULT TABLE

UpperResult
TECHBRIDGE

 STEP-BY-STEP LOGIC

1. Evaluates SQL function UPPER. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

ABS

 CONCEPT NOTES & DEFINITION

Absolute positive value

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ABS(X);
```

 SAMPLE INPUT DATA TABLE

AbsVal
42.5

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ABS(-42.5) AS AbsVal;
```

 EXECUTION OUTPUT RESULT TABLE

AbsVal
42.5

 STEP-BY-STEP LOGIC

1. Evaluates SQL function ABS. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

ACOS

 CONCEPT NOTES & DEFINITION

Arc cosine in radians

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ACOS(X);
```

 SAMPLE INPUT DATA TABLE

ArcCos
1.047197

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ACOS(0.5) AS ArcCos;
```

 EXECUTION OUTPUT RESULT TABLE

ArcCos
1.047197

 STEP-BY-STEP LOGIC

1. Evaluates SQL function ACOS. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

ASIN

 CONCEPT NOTES & DEFINITION

Arc sine in radians

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ASIN(X);
```

 SAMPLE INPUT DATA TABLE

ArcSin
0.523598

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ASIN(0.5) AS ArcSin;
```

 EXECUTION OUTPUT RESULT TABLE

ArcSin
0.523598

 STEP-BY-STEP LOGIC

1. Evaluates SQL function ASIN. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

ATAN

 CONCEPT NOTES & DEFINITION

Arc tangent in radians

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ATAN(X);
```

 SAMPLE INPUT DATA TABLE

ArcTan
0.785398

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ATAN(1) AS ArcTan;
```

 EXECUTION OUTPUT RESULT TABLE

ArcTan
0.785398

 STEP-BY-STEP LOGIC

1. Evaluates SQL function ATAN. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

ATAN2

 CONCEPT NOTES & DEFINITION

Arc tangent of two variables

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ATAN2(Y, X);
```

 SAMPLE INPUT DATA TABLE

ArcTan2
0.785398

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ATAN2(2, 2) AS ArcTan2;
```

 EXECUTION OUTPUT RESULT TABLE

ArcTan2
0.785398

 STEP-BY-STEP LOGIC

1. Evaluates SQL function ATAN2. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

AVG

 CONCEPT NOTES & DEFINITION

Average mean value

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT AVG(col) FROM t;
```

 SAMPLE INPUT DATA TABLE

AvgSalary
\$75,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  AVG(salary) AS AvgSalary FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

AvgSalary
\$75,000.00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function AVG. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CEIL

 CONCEPT NOTES & DEFINITION

Smallest integer $\geq$ number

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CEIL(X);
```

 SAMPLE INPUT DATA TABLE

CeilVal
25

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CEIL(24.1) AS CeilVal;
```

 EXECUTION OUTPUT RESULT TABLE

CeilVal
25

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CEIL. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CEILING

 CONCEPT NOTES & DEFINITION

Synonym for CEIL

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CEILING(X);
```

 SAMPLE INPUT DATA TABLE

CeilVal
25

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CEILING(24.1) AS CeilVal;
```

 EXECUTION OUTPUT RESULT TABLE

CeilVal
25

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CEILING. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

COS

 CONCEPT NOTES & DEFINITION

Cosine of angle in radians

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT COS(X);
```

 SAMPLE INPUT DATA TABLE

CosVal
-1

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  COS(PI()) AS CosVal;
```

 EXECUTION OUTPUT RESULT TABLE

CosVal
-1

 STEP-BY-STEP LOGIC

1. Evaluates SQL function COS. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

COT

 CONCEPT NOTES & DEFINITION

Cotangent of angle in radians

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT COT(X);
```

 SAMPLE INPUT DATA TABLE

CotVal
0.642092

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  COT(1) AS CotVal;
```

 EXECUTION OUTPUT RESULT TABLE

CotVal
0.642092

 STEP-BY-STEP LOGIC

1. Evaluates SQL function COT. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

COUNT

 CONCEPT NOTES & DEFINITION

Count of returned rows

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT COUNT(col) FROM t;
```

 SAMPLE INPUT DATA TABLE

CountVal
150

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
COUNT(*) AS CountVal FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

CountVal
150

 STEP-BY-STEP LOGIC

1. Evaluates SQL function COUNT. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DEGREES

 CONCEPT NOTES & DEFINITION

Radians to degrees

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DEGREES(X);
```

 SAMPLE INPUT DATA TABLE

DegreesVal
180

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  DEGREES(PI()) AS DegreesVal;
```

 EXECUTION OUTPUT RESULT TABLE

DegreesVal
180

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DEGREES. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DIV

 CONCEPT NOTES & DEFINITION

Integer division

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT N DIV M;
```

 SAMPLE INPUT DATA TABLE

Quotient
3

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  17 DIV 5 AS Quotient;
```

 EXECUTION OUTPUT RESULT TABLE

Quotient
3

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DIV. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

EXP

 CONCEPT NOTES & DEFINITION

e raised to power of N

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT EXP(X);
```

 SAMPLE INPUT DATA TABLE

ExpVal
2.718281

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  EXP(1) AS ExpVal;
```

 EXECUTION OUTPUT RESULT TABLE

ExpVal
2.718281

 STEP-BY-STEP LOGIC

1. Evaluates SQL function EXP. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

FLOOR

 CONCEPT NOTES & DEFINITION

Largest integer <= number

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT FLOOR(X);
```

 SAMPLE INPUT DATA TABLE

FloorVal
24

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  FLOOR(24.9) AS FloorVal;
```

 EXECUTION OUTPUT RESULT TABLE

FloorVal
24

 STEP-BY-STEP LOGIC

1. Evaluates SQL function FLOOR. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

GREATEST

 CONCEPT NOTES & DEFINITION

Greatest value in list

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT GREATEST(a1, a2);
```

 SAMPLE INPUT DATA TABLE

GreatestVal
99

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  GREATEST(10, 45, 99) AS GreatestVal;
```

 EXECUTION OUTPUT RESULT TABLE

GreatestVal
99

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function GREATEST.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LEAST

 CONCEPT NOTES & DEFINITION

Smallest value in list

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LEAST(a1, a2);
```

 SAMPLE INPUT DATA TABLE

LeastVal
10

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LEAST(10, 45, 99) AS LeastVal;
```

 EXECUTION OUTPUT RESULT TABLE

LeastVal
10

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LEAST. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LN

 CONCEPT NOTES & DEFINITION

Natural logarithm (base e)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LN(X);
```

 SAMPLE INPUT DATA TABLE

LnVal1
0.693147

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LN(2) AS LnVal1;
```

 EXECUTION OUTPUT RESULT TABLE

LnVal1
0.693147

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LN. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LOG

 CONCEPT NOTES & DEFINITION

Natural log or base

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LOG(b, X);
```

 SAMPLE INPUT DATA TABLE

LogVal
3

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LOG(2, 8) AS LogVal;
```

 EXECUTION OUTPUT RESULT TABLE

LogVal
3

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LOG. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LOG10

 CONCEPT NOTES & DEFINITION

Base-10 logarithm

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LOG10(X);
```

 SAMPLE INPUT DATA TABLE

Log10Val
3

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LOG10(1000) AS Log10Val;
```

 EXECUTION OUTPUT RESULT TABLE

Log10Val
3

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LOG10. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LOG2

 CONCEPT NOTES & DEFINITION

Base-2 logarithm

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LOG2(X);
```

 SAMPLE INPUT DATA TABLE

Log2Val
8

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LOG2(256) AS Log2Val;
```

 EXECUTION OUTPUT RESULT TABLE

Log2Val
8

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LOG2. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MAX

 CONCEPT NOTES & DEFINITION

Maximum value in set

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MAX(col) FROM t;
```

 SAMPLE INPUT DATA TABLE

MaxSalary
\$165,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  MAX(salary) AS MaxSalary FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

MaxSalary
\$165,000.00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function MAX. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MIN

 CONCEPT NOTES & DEFINITION

Minimum value in set

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MIN(col) FROM t;
```

 SAMPLE INPUT DATA TABLE

MinSalary
\$45,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  MIN(salary) AS MinSalary FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

MinSalary
\$45,000.00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function MIN. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MOD

 CONCEPT NOTES & DEFINITION

Division remainder

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MOD(N, M);
```

 SAMPLE INPUT DATA TABLE

ModVal
5

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  MOD(29, 6) AS ModVal;
```

 EXECUTION OUTPUT RESULT TABLE

ModVal
5

 STEP-BY-STEP LOGIC

1. Evaluates SQL function MOD. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

PI

 CONCEPT NOTES & DEFINITION

Value of Pi constant

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT PI();
```

 SAMPLE INPUT DATA TABLE

PiVal
3.141593

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  PI() AS PiVal;
```

 EXECUTION OUTPUT RESULT TABLE

PiVal
3.141593

 STEP-BY-STEP LOGIC

1. Evaluates SQL function PI. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

POW

 CONCEPT NOTES & DEFINITION

Power exponent calculation

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT POW(X, Y);
```

 SAMPLE INPUT DATA TABLE

PowVal
1024

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    POW(2, 10) AS PowVal;
```

 EXECUTION OUTPUT RESULT TABLE

PowVal
1024

 STEP-BY-STEP LOGIC

1. Evaluates SQL function POW. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

POWER

 CONCEPT NOTES & DEFINITION

Synonym for POW

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT POWER(X, Y);
```

 SAMPLE INPUT DATA TABLE

PowerVal
81

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    POWER(3, 4) AS PowerVal;
```

 EXECUTION OUTPUT RESULT TABLE

PowerVal
81

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function POWER.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

RADIANS

 CONCEPT NOTES & DEFINITION

Degrees to radians

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT RADIANS(X);
```

 SAMPLE INPUT DATA TABLE

RadVal
3.141592

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  RADIANS(180) AS RadVal;
```

 EXECUTION OUTPUT RESULT TABLE

RadVal
3.141592

 STEP-BY-STEP LOGIC

1. Evaluates SQL function RADIANS. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

RAND

 CONCEPT NOTES & DEFINITION

Random float between 0 & 1

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT RAND();
```

 SAMPLE INPUT DATA TABLE

RandVal
0.748291

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  RAND() AS RandVal;
```

 EXECUTION OUTPUT RESULT TABLE

RandVal
0.748291

 STEP-BY-STEP LOGIC

1. Evaluates SQL function RAND. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

ROUND

 CONCEPT NOTES & DEFINITION

Rounds to decimal places

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ROUND(X, D);
```

 SAMPLE INPUT DATA TABLE

RoundVal1
123.46

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ROUND(123.4567, 2) AS RoundVal1;
```

 EXECUTION OUTPUT RESULT TABLE

RoundVal1
123.46

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function ROUND.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SIGN

 CONCEPT NOTES & DEFINITION

Sign of number (-1, 0, 1)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SIGN(X);
```

 SAMPLE INPUT DATA TABLE

SignVal
-1

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SIGN(-50) AS SignVal;
```

 EXECUTION OUTPUT RESULT TABLE

SignVal
-1

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SIGN. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SIN

 CONCEPT NOTES & DEFINITION

Sine of angle in radians

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SIN(X);
```

 SAMPLE INPUT DATA TABLE

SinVal
1

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SIN(PI()/2) AS SinVal;
```

 EXECUTION OUTPUT RESULT TABLE

SinVal
1

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SIN. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SQRT

 CONCEPT NOTES & DEFINITION

Square root of number

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SQRT(X);
```

 SAMPLE INPUT DATA TABLE

SqrtVal
12

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SQRT(144) AS SqrtVal;
```

 EXECUTION OUTPUT RESULT TABLE

SqrtVal
12

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SQRT. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SUM

 CONCEPT NOTES & DEFINITION

Sum of set of values

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SUM(col) FROM t;
```

 SAMPLE INPUT DATA TABLE

SumSalary
\$11,250,000.00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    SUM(salary) AS SumSalary FROM employees;
```

 EXECUTION OUTPUT RESULT TABLE

SumSalary
\$11,250,000.00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SUM. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

TAN

 CONCEPT NOTES & DEFINITION

Tangent of angle in radians

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT TAN(X);
```

 SAMPLE INPUT DATA TABLE

TanVal
1

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  TAN(PI()/4) AS TanVal;
```

 EXECUTION OUTPUT RESULT TABLE

TanVal
1

 STEP-BY-STEP LOGIC

1. Evaluates SQL function TAN. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

TRUNCATE

 CONCEPT NOTES & DEFINITION

Truncates decimal places

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT TRUNCATE(X, D);
```

 SAMPLE INPUT DATA TABLE

TruncVal
123.45

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  TRUNCATE(123.4567, 2) AS TruncVal;
```

 EXECUTION OUTPUT RESULT TABLE

TruncVal
123.45

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function TRUNCATE.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

ADDDATE

 CONCEPT NOTES & DEFINITION

Adds interval to date

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ADDDATE(d, INTERVAL expr unit);
```

 SAMPLE INPUT DATA TABLE

AddDateVal
2026-10-08

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ADDDATE('2026-09-28', INTERVAL 10 DAY) AS AddDateVal;
```

 EXECUTION OUTPUT RESULT TABLE

AddDateVal
2026-10-08

 STEP-BY-STEP LOGIC

1. Evaluates SQL function ADDDATE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

ADDTIME

 CONCEPT NOTES & DEFINITION

Adds time interval to datetime

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ADDTIME(dt, t);
```

 SAMPLE INPUT DATA TABLE

AddTimeVal

2026-09-28 12:30:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ADDTIME('2026-09-28 10:00:00', '02:30:00') AS AddTimeVal;
```

 EXECUTION OUTPUT RESULT TABLE

AddTimeVal

2026-09-28 12:30:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function ADDTIME. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CURDATE

 CONCEPT NOTES & DEFINITION

Returns current server date

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CURDATE();
```

 SAMPLE INPUT DATA TABLE

CurDateVal
2026-09-28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CURDATE() AS CurDateVal;
```

 EXECUTION OUTPUT RESULT TABLE

CurDateVal
2026-09-28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CURDATE. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

CURRENT_DATE

 CONCEPT NOTES & DEFINITION

Synonym for CURDATE

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CURRENT_DATE();
```

 SAMPLE INPUT DATA TABLE

CurDateVal
2026-09-28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CURRENT_DATE() AS CurDateVal;
```

 EXECUTION OUTPUT RESULT TABLE

CurDateVal
2026-09-28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CURRENT_DATE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CURRENT_TIME

 CONCEPT NOTES & DEFINITION

Synonym for CURTIME

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CURRENT_TIME();
```

 SAMPLE INPUT DATA TABLE

CurTimeVal
00:43:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CURRENT_TIME() AS CurTimeVal;
```

 EXECUTION OUTPUT RESULT TABLE

CurTimeVal
00:43:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CURRENT_TIME. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CURRENT_TIMESTAMP

 CONCEPT NOTES & DEFINITION

Current date and time

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CURRENT_TIMESTAMP();
```

 SAMPLE INPUT DATA TABLE

CurTSVa1

2026-09-28 00:43:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CURRENT_TIMESTAMP() AS CurTSVa1;
```

 EXECUTION OUTPUT RESULT TABLE

CurTSVa1

2026-09-28 00:43:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CURRENT_TIMESTAMP. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CURTIME

 CONCEPT NOTES & DEFINITION

Returns current time

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CURTIME();
```

 SAMPLE INPUT DATA TABLE

CurTimeVal
00:43:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CURTIME() AS CurTimeVal;
```

 EXECUTION OUTPUT RESULT TABLE

CurTimeVal
00:43:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CURTIME. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DATE

 CONCEPT NOTES & DEFINITION

Extracts date portion

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DATE(dt);
```

 SAMPLE INPUT DATA TABLE

DateVal
2026-09-28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    DATE('2026-09-28 00:43:00') AS DateVal;
```

 EXECUTION OUTPUT RESULT TABLE

DateVal
2026-09-28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DATE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DATEDIFF

 CONCEPT NOTES & DEFINITION

Days difference between dates

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DATEDIFF(d1, d2);
```

 SAMPLE INPUT DATA TABLE

DaysDiff
94

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    DATEDIFF('2026-12-31', '2026-09-28') AS DaysDiff;
```

 EXECUTION OUTPUT RESULT TABLE

DaysDiff
94

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function DATEDIFF.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DATE_ADD

 CONCEPT NOTES & DEFINITION

Adds interval to date

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DATE_ADD(d, INTERVAL expr unit);
```

 SAMPLE INPUT DATA TABLE

DateAddVal

2026-10-28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  DATE_ADD('2026-09-28', INTERVAL 1 MONTH) AS DateAddVal;
```

 EXECUTION OUTPUT RESULT TABLE

DateAddVal

2026-10-28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DATE_ADD. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DATE_FORMAT

 CONCEPT NOTES & DEFINITION

Formats date by pattern

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DATE_FORMAT(d, format);
```

 SAMPLE INPUT DATA TABLE

FmtDate
Monday, September 28, 2026

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  DATE_FORMAT('2026-09-28', '%W, %M %d, %Y') AS FmtDate;
```

 EXECUTION OUTPUT RESULT TABLE

FmtDate
Monday, September 28, 2026

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function DATE_FORMAT.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DATE_SUB

 CONCEPT NOTES & DEFINITION

Subtracts interval from date

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DATE_SUB(d, INTERVAL expr unit);
```

 SAMPLE INPUT DATA TABLE

DateSubVal
2026-09-21

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  DATE_SUB('2026-09-28', INTERVAL 7 DAY) AS DateSubVal;
```

 EXECUTION OUTPUT RESULT TABLE

DateSubVal
2026-09-21

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function DATE_SUB.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DAY

 CONCEPT NOTES & DEFINITION

Day of month (1–31)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DAY(d);
```

 SAMPLE INPUT DATA TABLE

DayVal
28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    DAY('2026-09-28') AS DayVal;
```

 EXECUTION OUTPUT RESULT TABLE

DayVal
28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DAY. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DAYNAME

 CONCEPT NOTES & DEFINITION

Name of weekday

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DAYNAME(d);
```

 SAMPLE INPUT DATA TABLE

DayNameVal
Monday

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    DAYNAME('2026-09-28') AS DayNameVal;
```

 EXECUTION OUTPUT RESULT TABLE

DayNameVal
Monday

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DAYNAME. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

DAYOFMONTH

 CONCEPT NOTES & DEFINITION

Day of month (1–31)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DAYOFMONTH(d);
```

 SAMPLE INPUT DATA TABLE

DOMVa1
28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  DAYOFMONTH('2026-09-28') AS DOMVa1;
```

 EXECUTION OUTPUT RESULT TABLE

DOMVa1
28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DAYOFMONTH. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DAYOFWEEK

 CONCEPT NOTES & DEFINITION

Weekday index (1=Sun...7=Sat)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DAYOFWEEK(d);
```

 SAMPLE INPUT DATA TABLE

DOWVal
2

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  DAYOFWEEK('2026-09-28') AS DOWVal;
```

 EXECUTION OUTPUT RESULT TABLE

DOWVal
2

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DAYOFWEEK. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DAYOFYEAR

 CONCEPT NOTES & DEFINITION

Day of year (1–366)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DAYOFYEAR(d);
```

 SAMPLE INPUT DATA TABLE

DOYVa1
271

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  DAYOFYEAR('2026-09-28') AS DOYVa1;
```

 EXECUTION OUTPUT RESULT TABLE

DOYVa1
271

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DAYOFYEAR. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

EXTRACT

 CONCEPT NOTES & DEFINITION

Extracts part from date

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT EXTRACT(unit FROM d);
```

 SAMPLE INPUT DATA TABLE

ExtractedVal
9

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  EXTRACT(MONTH FROM '2026-09-28') AS ExtractedVal;
```

 EXECUTION OUTPUT RESULT TABLE

ExtractedVal
9

 STEP-BY-STEP LOGIC

1. Evaluates SQL function EXTRACT. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

FROM_DAYS

 CONCEPT NOTES & DEFINITION

Converts day count to date

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT FROM_DAYS(N);
```

 SAMPLE INPUT DATA TABLE

DateFromDays
2020-08-11

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  FROM_DAYS(738000) AS DateFromDays;
```

 EXECUTION OUTPUT RESULT TABLE

DateFromDays
2020-08-11

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function FROM_DAYS.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

HOUR

★
CONCEPT NOTES & DEFINITION

Hour part (0-23)

⚙️
FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT HOUR(t);
```

📊
SAMPLE INPUT DATA TABLE

HourVal
0

💻
EXECUTED PRACTICAL SQL QUERY

```
SELECT
    HOUR('00:43:00') AS HourVal;
```

📋
EXECUTION OUTPUT RESULT TABLE

HourVal
0

💡
STEP-BY-STEP LOGIC

1. Evaluates SQL function HOUR. 2. Performs calculation and returns result set.

⚠️
PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LAST_DAY

 CONCEPT NOTES & DEFINITION

Last day of month for date

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LAST_DAY(d);
```

 SAMPLE INPUT DATA TABLE

LastDayVal
2026-02-28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LAST_DAY('2026-02-15') AS LastDayVal;
```

 EXECUTION OUTPUT RESULT TABLE

LastDayVal
2026-02-28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LAST_DAY. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

LOCALTIME

 CONCEPT NOTES & DEFINITION

Current datetime

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LOCALTIME();
```

 SAMPLE INPUT DATA TABLE

LocalTimeVal
2026-09-28 00:43:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LOCALTIME() AS LocalTimeVal;
```

 EXECUTION OUTPUT RESULT TABLE

LocalTimeVal
2026-09-28 00:43:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LOCALTIME. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LOCALTIMESTAMP

 CONCEPT NOTES & DEFINITION

Current datetime

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LOCALTIMESTAMP();
```

 SAMPLE INPUT DATA TABLE

LocalTSVal
2026-09-28 00:43:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LOCALTIMESTAMP() AS LocalTSVal;
```

 EXECUTION OUTPUT RESULT TABLE

LocalTSVal
2026-09-28 00:43:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LOCALTIMESTAMP.
2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MAKEDATE

 CONCEPT NOTES & DEFINITION

Date from year & day of year

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MAKEDATE(yr, doy);
```

 SAMPLE INPUT DATA TABLE

MakedDate
2026-09-28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    MAKEDATE(2026, 271) AS MakedDate;
```

 EXECUTION OUTPUT RESULT TABLE

MakedDate
2026-09-28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function MAKEDATE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MAKETIME

 CONCEPT NOTES & DEFINITION

Time from hr, min, sec

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MAKETIME(h, m, s);
```

 SAMPLE INPUT DATA TABLE

MakedTime
14:30:45

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    MAKETIME(14, 30, 45) AS MakedTime;
```

 EXECUTION OUTPUT RESULT TABLE

MakedTime
14:30:45

 STEP-BY-STEP LOGIC

1. Evaluates SQL function MAKETIME. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MICROSECOND

 CONCEPT NOTES & DEFINITION

Microseconds part

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MICROSECOND(dt);
```

 SAMPLE INPUT DATA TABLE

MSVa1

123456

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    MICROSECOND('2026-09-28 00:43:00.123456') AS MSVa1;
```

 EXECUTION OUTPUT RESULT TABLE

MSVa1

123456

 STEP-BY-STEP LOGIC

1. Evaluates SQL function MICROSECOND. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MINUTE

 CONCEPT NOTES & DEFINITION

Minute part (0–59)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MINUTE(t);
```

 SAMPLE INPUT DATA TABLE

MinVal
43

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  MINUTE('00:43:00') AS MinVal;
```

 EXECUTION OUTPUT RESULT TABLE

MinVal
43

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function MINUTE.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MONTH

 CONCEPT NOTES & DEFINITION

Month number (1-12)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MONTH(d);
```

 SAMPLE INPUT DATA TABLE

MonthVal
9

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    MONTH('2026-09-28') AS MonthVal;
```

 EXECUTION OUTPUT RESULT TABLE

MonthVal
9

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function MONTH.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

MONTHNAME

 CONCEPT NOTES & DEFINITION

Month name

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT MONTHNAME(d);
```

 SAMPLE INPUT DATA TABLE

MonthNameVal

September

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    MONTHNAME('2026-09-28') AS MonthNameVal;
```

 EXECUTION OUTPUT RESULT TABLE

MonthNameVal

September

 STEP-BY-STEP LOGIC

1. Evaluates SQL function MONTHNAME. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

NOW

 CONCEPT NOTES & DEFINITION

Current date and time

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT NOW();
```

 SAMPLE INPUT DATA TABLE

NowVa1

2026-09-28 00:43:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  NOW() AS NowVa1;
```

 EXECUTION OUTPUT RESULT TABLE

NowVa1

2026-09-28 00:43:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function NOW. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

PERIOD_ADD

 CONCEPT NOTES & DEFINITION

Adds months to YYYYMM period

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT PERIOD_ADD(P, N);
```

 SAMPLE INPUT DATA TABLE

PeriodAddVal
202612

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  PERIOD_ADD(202609, 3) AS PeriodAddVal;
```

 EXECUTION OUTPUT RESULT TABLE

PeriodAddVal
202612

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function PERIOD_ADD.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

PERIOD_DIFF

 CONCEPT NOTES & DEFINITION

Diff between YYYYMM periods

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT PERIOD_DIFF(p1, p2);
```

 SAMPLE INPUT DATA TABLE

PeriodDiffVal
3

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  PERIOD_DIFF(202612, 202609) AS PeriodDiffVal;
```

 EXECUTION OUTPUT RESULT TABLE

PeriodDiffVal
3

 STEP-BY-STEP LOGIC

1. Evaluates SQL function PERIOD_DIFF. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

QUARTER

 CONCEPT NOTES & DEFINITION

Quarter of year (1-4)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT QUARTER(d);
```

 SAMPLE INPUT DATA TABLE

QuarterVal
3

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  QUARTER('2026-09-28') AS QuarterVal;
```

 EXECUTION OUTPUT RESULT TABLE

QuarterVal
3

 STEP-BY-STEP LOGIC

1. Evaluates SQL function QUARTER. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SECOND

 CONCEPT NOTES & DEFINITION

Seconds part (0–59)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SECOND(t);
```

 SAMPLE INPUT DATA TABLE

SecVal
0

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SECOND('00:43:00') AS SecVal;
```

 EXECUTION OUTPUT RESULT TABLE

SecVal
0

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SECOND. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

SEC_TO_TIME

 CONCEPT NOTES & DEFINITION

Seconds to 'HH:MM:SS'

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SEC_TO_TIME(sec);
```

 SAMPLE INPUT DATA TABLE

TimeVal
01:01:01

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SEC_TO_TIME(3661) AS TimeVal;
```

 EXECUTION OUTPUT RESULT TABLE

TimeVal
01:01:01

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SEC_TO_TIME. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

STR_TO_DATE

 CONCEPT NOTES & DEFINITION

Parses string to date

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT STR_TO_DATE(str, format);
```

 SAMPLE INPUT DATA TABLE

ParsedDate
2026-09-28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  STR_TO_DATE('28,09,2026', '%d,%m,%Y') AS ParsedDate;
```

 EXECUTION OUTPUT RESULT TABLE

ParsedDate
2026-09-28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function STR_TO_DATE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SUBDATE

 CONCEPT NOTES & DEFINITION

Subtracts date interval

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SUBDATE(d, days);
```

 SAMPLE INPUT DATA TABLE

SubDateVal
2026-09-21

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SUBDATE('2026-09-28', 7) AS SubDateVal;
```

 EXECUTION OUTPUT RESULT TABLE

SubDateVal
2026-09-21

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SUBDATE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SUBTIME

 CONCEPT NOTES & DEFINITION

Subtracts time interval

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SUBTIME(dt, t);
```

 SAMPLE INPUT DATA TABLE

SubTimeVal
00:33:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SUBTIME('00:43:00', '00:10:00') AS SubTimeVal;
```

 EXECUTION OUTPUT RESULT TABLE

SubTimeVal
00:33:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SUBTIME. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

SYSDATE

 CONCEPT NOTES & DEFINITION

System date and time

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SYSDATE();
```

 SAMPLE INPUT DATA TABLE

SysDateVal

2026-09-28 00:43:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  SYSDATE() AS SysDateVal;
```

 EXECUTION OUTPUT RESULT TABLE

SysDateVal

2026-09-28 00:43:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SYSDATE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

TIME

 CONCEPT NOTES & DEFINITION

Extracts time portion

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT TIME(dt);
```

 SAMPLE INPUT DATA TABLE

TimeVal
00:43:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    TIME('2026-09-28 00:43:00') AS TimeVal;
```

 EXECUTION OUTPUT RESULT TABLE

TimeVal
00:43:00

 STEP-BY-STEP LOGIC

1. Evaluates SQL function TIME. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

TIME_FORMAT

 CONCEPT NOTES & DEFINITION

Formats time pattern

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT TIME_FORMAT(t, format);
```

 SAMPLE INPUT DATA TABLE

FmtTime
11:14 PM

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  TIME_FORMAT('23:14:00', '%h:%i %p') AS FmtTime;
```

 EXECUTION OUTPUT RESULT TABLE

FmtTime
11:14 PM

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function TIME_FORMAT.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

TIME_TO_SEC



CONCEPT NOTES & DEFINITION

Time converted to seconds



FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT TIME_TO_SEC(t);
```



SAMPLE INPUT DATA TABLE

SecVal
3661



EXECUTED PRACTICAL SQL QUERY

```
SELECT
    TIME_TO_SEC('01:01:01') AS SecVal;
```



EXECUTION OUTPUT RESULT TABLE

SecVal
3661



STEP-BY-STEP LOGIC

1. Evaluates SQL function TIME_TO_SEC. 2. Performs calculation and returns result set.



PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

TIMEDIFF

 CONCEPT NOTES & DEFINITION

Difference between two times

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT TIMEDIFF(t1, t2);
```

 SAMPLE INPUT DATA TABLE

TimeDiffVal
03:14:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  TIMEDIFF('23:14:00', '20:00:00') AS TimeDiffVal;
```

 EXECUTION OUTPUT RESULT TABLE

TimeDiffVal
03:14:00

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function TIMEDIFF.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

TIMESTAMP

 CONCEPT NOTES & DEFINITION

Returns datetime value

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT TIMESTAMP(d, t);
```

 SAMPLE INPUT DATA TABLE

TSVa1
2026-09-28 12:00:00

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  TIMESTAMP('2026-09-28', '12:00:00') AS TSVa1;
```

 EXECUTION OUTPUT RESULT TABLE

TSVa1
2026-09-28 12:00:00

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function TIMESTAMP.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

TO_DAYS

 CONCEPT NOTES & DEFINITION

Days since year 0

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT TO_DAYS(d);
```

 SAMPLE INPUT DATA TABLE

DaysVal
740252

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  TO_DAYS('2026-09-28') AS DaysVal;
```

 EXECUTION OUTPUT RESULT TABLE

DaysVal
740252

 STEP-BY-STEP LOGIC

1. Evaluates SQL function TO_DAYS. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

WEEK

 CONCEPT NOTES & DEFINITION

Week number of year

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT WEEK(d);
```

 SAMPLE INPUT DATA TABLE

WeekVal
39

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    WEEK('2026-09-28') AS WeekVal;
```

 EXECUTION OUTPUT RESULT TABLE

WeekVal
39

 STEP-BY-STEP LOGIC

1. Evaluates SQL function WEEK. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

WEEKDAY

 CONCEPT NOTES & DEFINITION

Weekday index (0=Mon..6=Sun)

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT WEEKDAY(d);
```

 SAMPLE INPUT DATA TABLE

WeekdayVal
0

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    WEEKDAY('2026-09-28') AS WeekdayVal;
```

 EXECUTION OUTPUT RESULT TABLE

WeekdayVal
0

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function WEEKDAY.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

WEEKOFYEAR

 CONCEPT NOTES & DEFINITION

Calendar week of year

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT WEEKOFYEAR(d);
```

 SAMPLE INPUT DATA TABLE

WOYVa1
40

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    WEEKOFYEAR('2026-09-28') AS WOYVa1;
```

 EXECUTION OUTPUT RESULT TABLE

WOYVa1
40

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function WEEKOFYEAR.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

YEAR



CONCEPT NOTES & DEFINITION

Year part of date



FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT YEAR(d);
```



SAMPLE INPUT DATA TABLE

YearVal
2026



EXECUTED PRACTICAL SQL QUERY

```
SELECT
  YEAR('2026-09-28') AS YearVal;
```



EXECUTION OUTPUT RESULT TABLE

YearVal
2026



STEP-BY-STEP LOGIC

1. Evaluates SQL function YEAR. 2. Performs calculation and returns result set.



PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

YEARWEEK

 CONCEPT NOTES & DEFINITION

Combined year and week

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT YEARWEEK(d);
```

 SAMPLE INPUT DATA TABLE

YearWeekVal
202639

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    YEARWEEK('2026-09-28') AS YearWeekVal;
```

 EXECUTION OUTPUT RESULT TABLE

YearWeekVal
202639

 STEP-BY-STEP LOGIC

1. Evaluates SQL function YEARWEEK. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

BIN

 CONCEPT NOTES & DEFINITION

Binary representation string

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT BIN(N);
```

 SAMPLE INPUT DATA TABLE

BinVal
1111

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  BIN(15) AS BinVal;
```

 EXECUTION OUTPUT RESULT TABLE

BinVal
1111

 STEP-BY-STEP LOGIC

1. Evaluates SQL function BIN. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

BINARY

 CONCEPT NOTES & DEFINITION

Casts to binary string

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT BINARY str;
```

 SAMPLE INPUT DATA TABLE

BinString
TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  BINARY 'TechBridge' AS BinString;
```

 EXECUTION OUTPUT RESULT TABLE

BinString
TechBridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function BINARY. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CASE

 CONCEPT NOTES & DEFINITION

Evaluates conditional branches

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CASE WHEN cond THEN res ELSE def END;
```

 SAMPLE INPUT DATA TABLE

CaseResult
Pass

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
CASE WHEN 10 > 5 THEN 'Pass' ELSE 'Fail' END AS CaseResult;
```

 EXECUTION OUTPUT RESULT TABLE

CaseResult
Pass

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CASE. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CAST

 CONCEPT NOTES & DEFINITION

Converts value to data type

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CAST(expr AS type);
```

 SAMPLE INPUT DATA TABLE

CastResult
2026-09-28

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CAST('2026-09-28' AS DATE) AS CastResult;
```

 EXECUTION OUTPUT RESULT TABLE

CastResult
2026-09-28

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CAST. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

COALESCE

 CONCEPT NOTES & DEFINITION

First non-NULL item in list

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT COALESCE(v1, v2, ...);
```

 SAMPLE INPUT DATA TABLE

CoalesceVal
TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  COALESCE(NULL, NULL, 'TechBridge') AS CoalesceVal;
```

 EXECUTION OUTPUT RESULT TABLE

CoalesceVal
TechBridge

 STEP-BY-STEP LOGIC

- 1. Evaluates SQL function COALESCE.
- 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CONNECTION_ID

 CONCEPT NOTES & DEFINITION

Unique session connection ID

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CONNECTION_ID();
```

 SAMPLE INPUT DATA TABLE

ConnID
1042

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CONNECTION_ID() AS ConnID;
```

 EXECUTION OUTPUT RESULT TABLE

ConnID
1042

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CONNECTION_ID.
2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CONV



CONCEPT NOTES & DEFINITION

Converts number between bases



FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CONV(N, from, to);
```



SAMPLE INPUT DATA TABLE

ConvVal

1111



EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CONV(15, 10, 2) AS ConvVal;
```



EXECUTION OUTPUT RESULT TABLE

ConvVal

1111



STEP-BY-STEP LOGIC

1. Evaluates SQL function CONV. 2. Performs calculation and returns result set.



PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

CONVERT

 CONCEPT NOTES & DEFINITION

Converts data type or charset

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CONVERT(expr, type);
```

 SAMPLE INPUT DATA TABLE

ConvertedVal
TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CONVERT('TechBridge', CHAR) AS ConvertedVal;
```

 EXECUTION OUTPUT RESULT TABLE

ConvertedVal
TechBridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CONVERT. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

CURRENT_USER

 CONCEPT NOTES & DEFINITION

Authenticated user & host

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT CURRENT_USER();
```

 SAMPLE INPUT DATA TABLE

CurUser

techadmin@localhost

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  CURRENT_USER() AS CurUser;
```

 EXECUTION OUTPUT RESULT TABLE

CurUser

techadmin@localhost

 STEP-BY-STEP LOGIC

1. Evaluates SQL function CURRENT_USER. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

DATABASE

 CONCEPT NOTES & DEFINITION

Current active database name

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT DATABASE();
```

 SAMPLE INPUT DATA TABLE

DBName
techbridge_db

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  DATABASE() AS DBName;
```

 EXECUTION OUTPUT RESULT TABLE

DBName
techbridge_db

 STEP-BY-STEP LOGIC

1. Evaluates SQL function DATABASE. 2.
Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL
inputs gracefully.

IF

 CONCEPT NOTES & DEFINITION

Ternary evaluation

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT IF(cond, v1, v2);
```

 SAMPLE INPUT DATA TABLE

IfVal
Pass

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  IF(500 > 100, 'Pass', 'Fail') AS IfVal;
```

 EXECUTION OUTPUT RESULT TABLE

IfVal
Pass

 STEP-BY-STEP LOGIC

1. Evaluates SQL function IF. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

IFNULL

 CONCEPT NOTES & DEFINITION

Returns val1 if not NULL else val2

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT IFNULL(v1, v2);
```

 SAMPLE INPUT DATA TABLE

IfNullVal
Default

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  IFNULL(NULL, 'Default') AS IfNullVal;
```

 EXECUTION OUTPUT RESULT TABLE

IfNullVal
Default

 STEP-BY-STEP LOGIC

1. Evaluates SQL function IFNULL. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

ISNULL

 CONCEPT NOTES & DEFINITION

Returns 1 if NULL else 0

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT ISNULL(expr);
```

 SAMPLE INPUT DATA TABLE

IsNullVal
1

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  ISNULL(NULL) AS IsNullVal;
```

 EXECUTION OUTPUT RESULT TABLE

IsNullVal
1

 STEP-BY-STEP LOGIC

1. Evaluates SQL function ISNULL. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

LAST_INSERT_ID

 CONCEPT NOTES & DEFINITION

Auto-increment ID of last row

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT LAST_INSERT_ID();
```

 SAMPLE INPUT DATA TABLE

LastID
1089

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  LAST_INSERT_ID() AS LastID;
```

 EXECUTION OUTPUT RESULT TABLE

LastID
1089

 STEP-BY-STEP LOGIC

1. Evaluates SQL function LAST_INSERT_ID. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

NULLIF

 CONCEPT NOTES & DEFINITION

Returns NULL if expr1 equals expr2

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT NULLIF(e1, e2);
```

 SAMPLE INPUT DATA TABLE

NullIfVal
NULL

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  NULLIF('Tech', 'Tech') AS NullIfVal;
```

 EXECUTION OUTPUT RESULT TABLE

NullIfVal
NULL

 STEP-BY-STEP LOGIC

1. Evaluates SQL function NULLIF. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SESSION_USER

 CONCEPT NOTES & DEFINITION

Current session username

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SESSION_USER();
```

 SAMPLE INPUT DATA TABLE

SessUser
techadmin@localhost

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    SESSION_USER() AS SessUser;
```

 EXECUTION OUTPUT RESULT TABLE

SessUser
techadmin@localhost

 STEP-BY-STEP LOGIC

1. Evaluates SQL function SESSION_USER. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

SYSTEM_USER

✦ CONCEPT NOTES & DEFINITION

Current system username

⚙️ FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT SYSTEM_USER();
```

📊 SAMPLE INPUT DATA TABLE

SysUser
techadmin@localhost

💻 EXECUTED PRACTICAL SQL QUERY

```
SELECT
    SYSTEM_USER() AS SysUser;
```

📋 EXECUTION OUTPUT RESULT TABLE

SysUser
techadmin@localhost

💡 STEP-BY-STEP LOGIC

- Evaluates SQL function SYSTEM_USER.
- Performs calculation and returns result set.

⚠️ PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

USER

 CONCEPT NOTES & DEFINITION

Client username and host

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT USER();
```

 SAMPLE INPUT DATA TABLE

UserVal

techadmin@localhost

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  USER() AS UserVal;
```

 EXECUTION OUTPUT RESULT TABLE

UserVal

techadmin@localhost

 STEP-BY-STEP LOGIC

1. Evaluates SQL function USER. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.

VERSION

 CONCEPT NOTES & DEFINITION

MySQL server version string

 FUNCTION / COMMAND SYNTAX

```
-- FUNCTION SYNTAX
SELECT VERSION();
```

 SAMPLE INPUT DATA TABLE

VerVal
8.0.35-TechBridge

 EXECUTED PRACTICAL SQL QUERY

```
SELECT
  VERSION() AS VerVal;
```

 EXECUTION OUTPUT RESULT TABLE

VerVal
8.0.35-TechBridge

 STEP-BY-STEP LOGIC

1. Evaluates SQL function VERSION. 2. Performs calculation and returns result set.

 PRO TIP & PITFALL

Verify column data types and handles NULL inputs gracefully.